

```
In [ ]: # convert jupyter notebook to html file
! jupyter nbconvert --to html /content/KadamOjasDSCI303Final.ipynb
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [ ]:
```

```
In [ ]: import pandas as pd
import numpy as np

path = "/content/drive/MyDrive/Final_Augmented_dataset_Diseases_and_Symptoms"
df = pd.read_csv(path)

df.head()
```

```
Out[ ]:
```

	diseases	anxiety and nervousness	depression	shortness of breath	depressive or psychotic symptoms	sharp chest pain	dizziness	inson
0	panic disorder	1	0	1	1	0	0	
1	panic disorder	0	0	1	1	0	1	
2	panic disorder	1	1	1	1	0	1	
3	panic disorder	1	0	0	1	0	1	
4	panic disorder	1	1	0	0	0	0	

5 rows x 378 columns

```
In [ ]: df.shape
```

```
Out[ ]: (246945, 379)
```

```
In [ ]: print("Shape:", df.shape)
print("Number of unique diseases:", df["diseases"].nunique())
print("\nFirst 10 columns:")
print(df.columns[:10].tolist())

print("\nHead:")
print(df.head())
```

```
print("\nMissing values:")  
print(df.isnull().sum().sum())
```

Shape: (246945, 378)

Number of unique diseases: 773

First 10 columns:

['diseases', 'anxiety and nervousness', 'depression', 'shortness of breath', 'depressive or psychotic symptoms', 'sharp chest pain', 'dizziness', 'insomnia', 'abnormal involuntary movements', 'chest tightness']

Head:

	diseases	anxiety and nervousness	depression	shortness of breath	
\					
0	panic disorder	1	0		1
1	panic disorder	0	0		1
2	panic disorder	1	1		1
3	panic disorder	1	0		0
4	panic disorder	1	1		0
	depressive or psychotic symptoms		sharp chest pain	dizziness	insomnia
\					
0		1	0	0	0
1		1	0	1	1
2		1	0	1	1
3		1	0	1	1
4		0	0	0	1
	abnormal involuntary movements		chest tightness	...	\
0		0	1	...	
1		0	0	...	
2		0	0	...	
3		1	0	...	
4		1	1	...	
	stuttering or stammering		problems with orgasm	nose deformity	\
0		0	0	0	
1		0	0	0	
2		0	0	0	
3		0	0	0	
4		0	0	0	
	lump over jaw	sore in nose	hip weakness	back swelling	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
	ankle stiffness or tightness		ankle weakness	neck weakness	
0		0	0	0	
1		0	0	0	
2		0	0	0	
3		0	0	0	
4		0	0	0	

[5 rows x 378 columns]

Missing values:

0

```
In [ ]: # =====
# Basic EDA
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Shape:", df.shape)
print("Unique diseases:", df["diseases"].nunique())
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())

# class distribution
disease_counts = df["diseases"].value_counts()
print("\nTop 10 diseases:")
print(disease_counts.head(10))

# number of symptoms per sample
symptom_cols = df.columns.drop("diseases")
df["num_symptoms"] = df[symptom_cols].sum(axis=1)

print("\nSymptoms per sample summary:")
print(df["num_symptoms"].describe())

# most common symptoms
symptom_freq = df[symptom_cols].sum().sort_values(ascending=False)
print("\nTop 10 most common symptoms:")
print(symptom_freq.head(10))

# plot top diseases
plt.figure(figsize=(10,5))
disease_counts.head(10).plot(kind="bar")
plt.title("Top 10 Most Common Diseases")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()

# plot symptoms per sample
plt.figure(figsize=(8,5))
plt.hist(df["num_symptoms"], bins=30)
plt.title("Number of Symptoms per Sample")
plt.xlabel("Number of Symptoms")
plt.ylabel("Count")
plt.show()
```

Shape: (246945, 378)
Unique diseases: 773
Missing values: 0
Duplicate rows: 57298

Top 10 diseases:

diseases	
cystitis	1219
nose disorder	1218
vulvodynia	1218
complex regional pain syndrome	1217
spondylosis	1216
vaginal cyst	1215
esophagitis	1215
peripheral nerve disorder	1215
hypoglycemia	1215
conjunctivitis due to allergy	1215

Name: count, dtype: int64

Symptoms per sample summary:

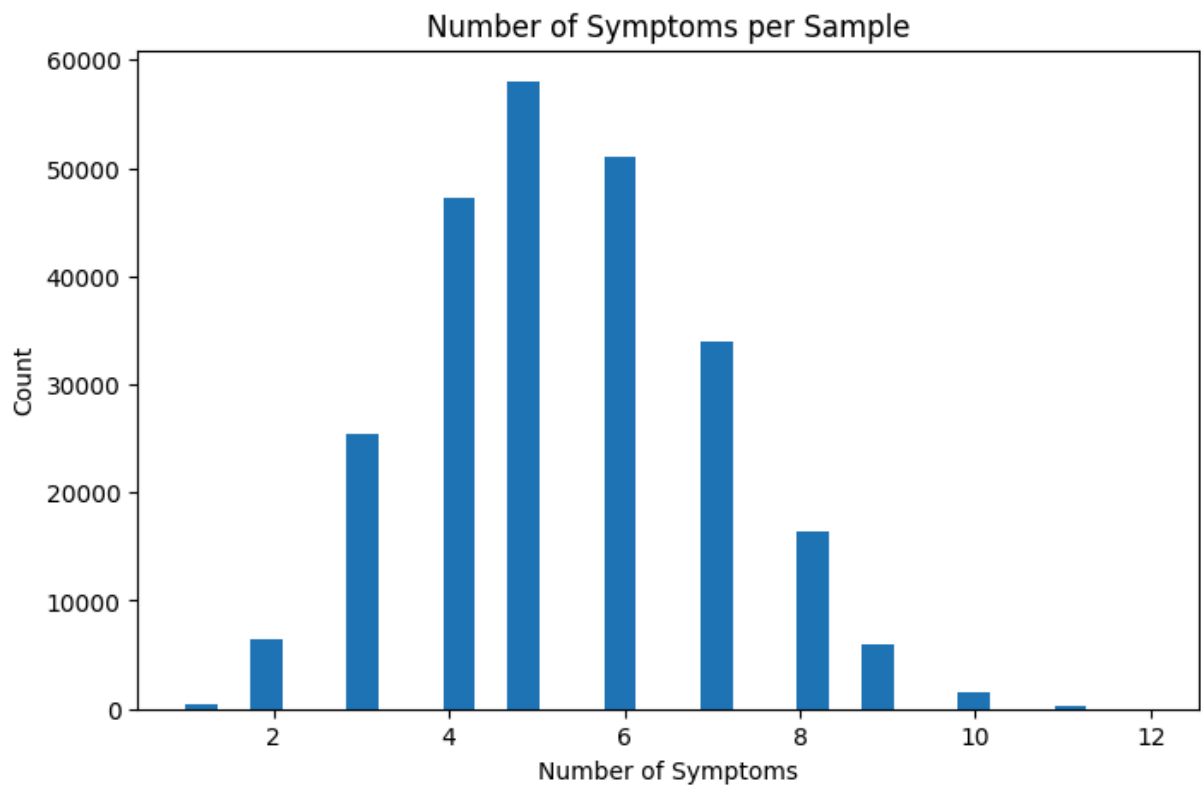
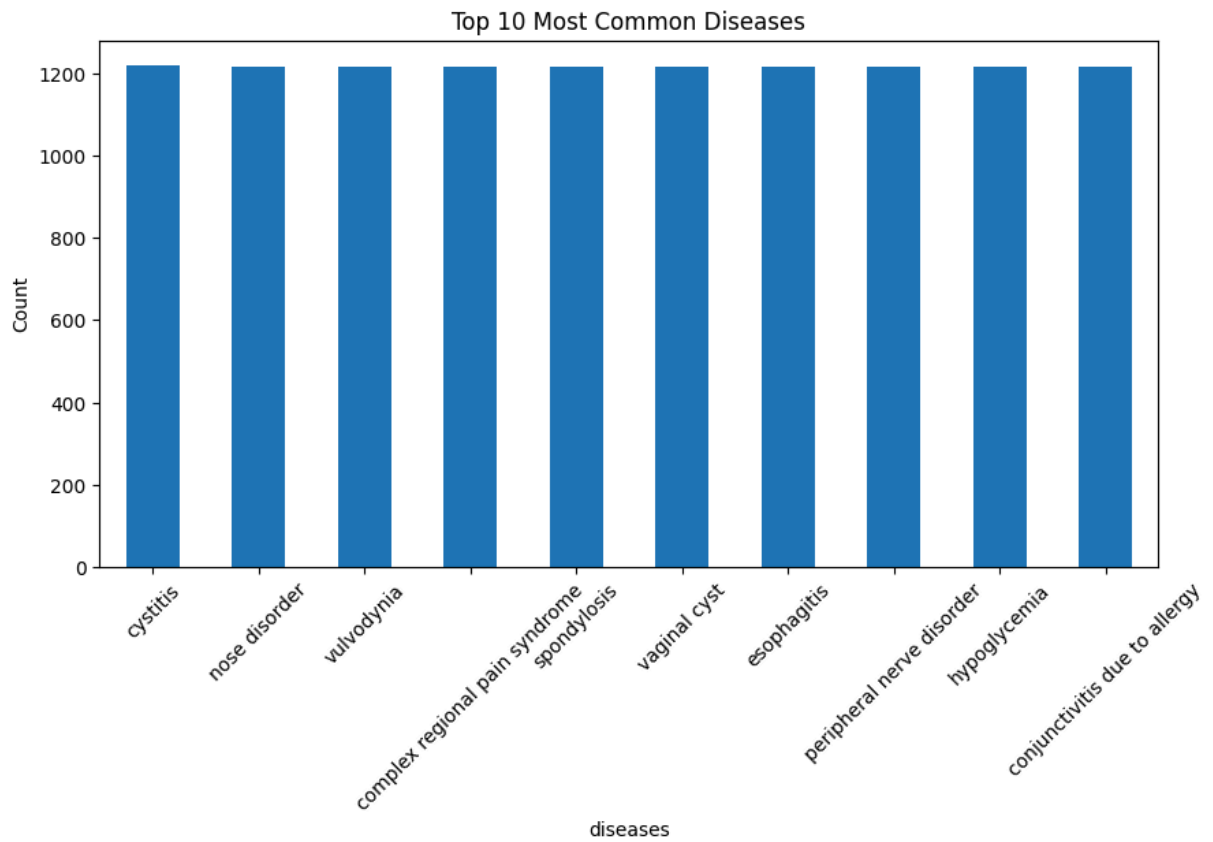
count	246945.000000
mean	5.332851
std	1.640610
min	1.000000
25%	4.000000
50%	5.000000
75%	6.000000
max	12.000000

Name: num_symptoms, dtype: float64

Top 10 most common symptoms:

sharp abdominal pain	32307
vomiting	27874
headache	24719
cough	24296
sharp chest pain	24016
nausea	23687
back pain	21809
shortness of breath	21346
fever	20394
dizziness	17272

dtype: int64



```
In [ ]: # -----
# 2. Basic overview
# -----
print("Shape of dataset:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
```

```

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

# -----
# 3. Identify label column
# -----
if "diseases" in df.columns:
    label_col = "diseases"
elif "Disease" in df.columns:
    label_col = "Disease"
else:
    label_col = df.columns[-1] # assume last column is target

print("\nLabel column:", label_col)

# -----
# 4. Missing values
# -----
missing_per_col = df.isnull().sum()
total_missing = missing_per_col.sum()

print("\nTotal missing values in dataset:", total_missing)
print("\nColumns with missing values:")
print(missing_per_col[missing_per_col > 0].sort_values(ascending=False))

# -----
# 5. Duplicate rows
# -----
num_duplicates = df.duplicated().sum()
print("\nNumber of duplicate rows:", num_duplicates)

# -----
# 6. Target variable analysis
# -----
print("\nNumber of unique diseases:", df[label_col].nunique())

class_counts = df[label_col].value_counts()

print("\nTop 10 most common diseases:")
print(class_counts.head(10))

print("\nTop 10 least common diseases:")
print(class_counts.tail(10))

print("\nSummary of class counts:")
print(class_counts.describe())

# -----
# 7. Feature analysis
# -----
X = df.drop(columns=[label_col])

```

```

print("\nNumber of feature columns:", X.shape[1])

# number of unique values in each feature
unique_counts = X.nunique()

print("\nFeature unique-value summary:")
print(unique_counts.describe())

# check if features are mostly binary
non_binary_cols = [col for col in X.columns if not set(X[col].dropna().unique())

print("\nNumber of non-binary feature columns:", len(non_binary_cols))
if len(non_binary_cols) > 0:
    print("Some non-binary columns:")
    print(non_binary_cols[:20])

# symptom frequency
symptom_counts = X.sum().sort_values(ascending=False)

print("\nTop 20 most frequent symptoms:")
print(symptom_counts.head(20))

print("\nTop 20 least frequent symptoms:")
print(symptom_counts.tail(20))

# -----
# 8. Rare classes
# -----
rare_classes = class_counts[class_counts < 10]
print("\nNumber of diseases with fewer than 2 samples:", len(rare_classes))

# optional
print("\nDiseases with fewer than 2 samples:")
print(rare_classes)

# -----
# 9. Quick plots
# -----

# class distribution
plt.figure(figsize=(10, 5))
class_counts.plot(kind="hist", bins=50)
plt.title("Distribution of Number of Samples per Disease")
plt.xlabel("Samples per Disease")
plt.ylabel("Number of Diseases")
plt.show()

# top 20 diseases
plt.figure(figsize=(12, 6))
class_counts.head(20).plot(kind="bar")
plt.title("Top 20 Most Common Diseases")
plt.xlabel("Disease")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()

```



```

# top 20 symptoms
plt.figure(figsize=(12, 6))
symptom_counts.head(20).plot(kind="bar")
plt.title("Top 20 Most Frequent Symptoms")
plt.xlabel("Symptom")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()

# -----
# 10. Useful summary prints
# -----
print("\nFinal EDA Summary")
print("-----")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Features:", X.shape[1])
print("Unique diseases:", df[label_col].nunique())
print("Total missing values:", total_missing)
print("Duplicate rows:", num_duplicates)
print("Diseases with <2 samples:", len(rare_classes))
print("Non-binary feature columns:", len(non_binary_cols))

```

Shape of dataset: (246945, 379)

First 5 rows:

	diseases	anxiety and nervousness	depression	shortness of breath					
\									
0	panic disorder	1	0	1					
1	panic disorder	0	0	1					
2	panic disorder	1	1	1					
3	panic disorder	1	0	0					
4	panic disorder	1	1	0					
	depressive or psychotic symptoms		sharp chest pain	dizziness	insomnia				
\									
0		1	0	0	0				
1		1	0	1	1				
2		1	0	1	1				
3		1	0	1	1				
4		0	0	0	1				
	abnormal involuntary movements		chest tightness	...	problems with orgas				
m \									
0		0	1	...					
0									
1		0	0	...					
0									
2		0	0	...					
0									
3		1	0	...					
0									
4		1	1	...					
0									
	nose deformity	lump over jaw	sore in nose	hip weakness	back swelling				
\									
0	0	0	0	0	0				
1	0	0	0	0	0				
2	0	0	0	0	0				
3	0	0	0	0	0				
4	0	0	0	0	0				
	ankle stiffness or tightness	ankle weakness	neck weakness	num_symptoms					
0	0	0	0	7					
1	0	0	0	5					
2	0	0	0	9					
3	0	0	0	6					
4	0	0	0	5					

[5 rows x 379 columns]

Column names:

['diseases', 'anxiety and nervousness', 'depression', 'shortness of breath', 'depressive or psychotic symptoms', 'sharp chest pain', 'dizziness', 'insomnia', 'abnormal involuntary movements', 'chest tightness', 'palpitations', 'irregular heartbeat', 'breathing fast', 'hoarse voice', 'sore throat', 'difficulty speaking', 'cough', 'nasal congestion', 'throat swelling', 'diminished hearing', 'lump in throat', 'throat feels tight', 'difficulty in swallowin

g', 'skin swelling', 'retention of urine', 'groin mass', 'leg pain', 'hip pain', 'suprapubic pain', 'blood in stool', 'lack of growth', 'emotional symptoms', 'elbow weakness', 'back weakness', 'pus in sputum', 'symptoms of the scrotum and testes', 'swelling of scrotum', 'pain in testicles', 'flatulence', 'pus draining from ear', 'jaundice', 'mass in scrotum', 'white discharge from eye', 'irritable infant', 'abusing alcohol', 'fainting', 'hostile behavior', 'drug abuse', 'sharp abdominal pain', 'feeling ill', 'vomiting', 'headache', 'nausea', 'diarrhea', 'vaginal itching', 'vaginal dryness', 'painful urination', 'involuntary urination', 'pain during intercourse', 'frequent urination', 'lower abdominal pain', 'vaginal discharge', 'blood in urine', 'hot flashes', 'intermenstrual bleeding', 'hand or finger pain', 'wrist pain', 'hand or finger swelling', 'arm pain', 'wrist swelling', 'arm stiffness or tightness', 'arm swelling', 'hand or finger stiffness or tightness', 'wrist stiffness or tightness', 'lip swelling', 'toothache', 'abnormal appearing skin', 'skin lesion', 'acne or pimples', 'dry lips', 'facial pain', 'mouth ulcer', 'skin growth', 'eye deviation', 'diminished vision', 'double vision', 'cross-eyed', 'symptoms of eye', 'pain in eye', 'eye moves abnormally', 'abnormal movement of eyelid', 'foreign body sensation in eye', 'irregular appearing scalp', 'swollen lymph nodes', 'back pain', 'neck pain', 'low back pain', 'pain of the anus', 'pain during pregnancy', 'pelvic pain', 'impotence', 'infant spitting up', 'vomiting blood', 'regurgitation', 'burning abdominal pain', 'restlessness', 'symptoms of infants', 'wheezing', 'peripheral edema', 'neck mass', 'ear pain', 'jaw swelling', 'mouth dryness', 'neck swelling', 'knee pain', 'foot or toe pain', 'bowlegged or knock-kneed', 'ankle pain', 'bones are painful', 'knee weakness', 'elbow pain', 'knee swelling', 'skin moles', 'knee lump or mass', 'weight gain', 'problems with movement', 'knee stiffness or tightness', 'leg swelling', 'foot or toe swelling', 'heartburn', 'smoking problems', 'muscle pain', 'infant feeding problem', 'recent weight loss', 'problems with shape or size of breast', 'underweight', 'difficulty eating', 'scanty menstrual flow', 'vaginal pain', 'vaginal redness', 'vulvar irritation', 'weakness', 'decreased heart rate', 'increased heart rate', 'bleeding or discharge from nipple', 'ringing in ear', 'plugged feeling in ear', 'itchy ear(s)', 'frontal headache', 'fluid in ear', 'neck stiffness or tightness', 'spots or clouds in vision', 'eye redness', 'lacrimation', 'itchiness of eye', 'blindness', 'eye burns or stings', 'itchy eyelid', 'feeling cold', 'decreased appetite', 'excessive appetite', 'excessive anger', 'loss of sensation', 'focal weakness', 'slurring words', 'symptoms of the face', 'disturbance of memory', 'paresthesia', 'side pain', 'fever', 'shoulder pain', 'shoulder stiffness or tightness', 'shoulder weakness', 'arm cramps or spasms', 'shoulder swelling', 'tongue lesions', 'leg cramps or spasms', 'abnormal appearing tongue', 'ache all over', 'lower body pain', 'problems during pregnancy', 'spotting or bleeding during pregnancy', 'cramps and spasms', 'upper abdominal pain', 'stomach bloating', 'changes in stool appearance', 'unusual color or odor to urine', 'kidney mass', 'swollen abdomen', 'symptoms of prostate', 'leg stiffness or tightness', 'difficulty breathing', 'rib pain', 'joint pain', 'muscle stiffness or tightness', 'pallor', 'hand or finger lump or mass', 'chills', 'groin pain', 'fatigue', 'abdominal distention', 'regurgitation.1', 'symptoms of the kidneys', 'melena', 'flushing', 'coughing up sputum', 'seizures', 'delusions or hallucinations', 'shoulder cramps or spasms', 'joint stiffness or tightness', 'pain or soreness of breast', 'excessive urination at night', 'bleeding from eye', 'rectal bleeding', 'constipation', 'temper problems', 'coryza', 'wrist weakness', 'eye strain', 'hemoptysis', 'lymphedema', 'skin on leg or foot looks infected', 'allergic reaction', 'congestion in chest', 'muscle swelling', 'pus in urine', 'abnormal size or shape of ear', 'low back weakness', 'sleepiness', 'apnea', 'abnormal breathing sounds', 'excessive growth', 'elbow cramps or spasms', 'feeling hot and col

d', 'blood clots during menstrual periods', 'absence of menstruation', 'pulling at ears', 'gum pain', 'redness in ear', 'fluid retention', 'flu-like syndrome', 'sinus congestion', 'painful sinuses', 'fears and phobias', 'recent pregnancy', 'uterine contractions', 'burning chest pain', 'back cramps or spasms', 'stiffness all over', 'muscle cramps, contractures, or spasms', 'low back cramps or spasms', 'back mass or lump', 'nosebleed', 'long menstrual periods', 'heavy menstrual flow', 'unpredictable menstruation', 'painful menstruation', 'infertility', 'frequent menstruation', 'sweating', 'mass on eyelid', 'swollen eye', 'eyelid swelling', 'eyelid lesion or rash', 'unwanted hair', 'symptoms of bladder', 'irregular appearing nails', 'itching of skin', 'hurts to breath', 'nailbiting', 'skin dryness, peeling, scaliness, or roughness', 'skin on arm or hand looks infected', 'skin irritation', 'itchy scalp', 'hip swelling', 'incontinence of stool', 'foot or toe cramps or spasms', 'warts', 'bumps on penis', 'too little hair', 'foot or toe lump or mass', 'skin rash', 'mass or swelling around the anus', 'low back swelling', 'ankle swelling', 'hip lump or mass', 'drainage in throat', 'dry or flaky scalp', 'premenstrual tension or irritability', 'feeling hot', 'feet turned in', 'foot or toe stiffness or tightness', 'pelvic pressure', 'elbow swelling', 'elbow stiffness or tightness', 'early or late onset of menopause', 'mass on ear', 'bleeding from ear', 'hand or finger weakness', 'low self-esteem', 'throat irritation', 'itching of the anus', 'swollen or red tonsils', 'irregular belly button', 'swollen tongue', 'lip sore', 'vulvar sore', 'hip stiffness or tightness', 'mouth pain', 'arm weakness', 'leg lump or mass', 'disturbance of smell or taste', 'discharge in stools', 'penis pain', 'loss of sex drive', 'obsessions and compulsions', 'antisocial behavior', 'neck cramps or spasms', 'pupils unequal', 'poor circulation', 'thirst', 'sleepwalking', 'skin oiliness', 'sneezing', 'bladder mass', 'knee cramps or spasms', 'premature ejaculation', 'leg weakness', 'posture problems', 'bleeding in mouth', 'tongue bleeding', 'change in skin mole size or color', 'penis redness', 'penile discharge', 'shoulder lump or mass', 'polyuria', 'cloudy eye', 'hysterical behavior', 'arm lump or mass', 'nightmares', 'bleeding gums', 'pain in gums', 'bedwetting', 'diaper rash', 'lump or mass of breast', 'vaginal bleeding after menopause', 'infrequent menstruation', 'mass on vulva', 'jaw pain', 'itching of scrotum', 'postpartum problems of the breast', 'eyelid retracted', 'hesitancy', 'elbow lump or mass', 'muscle weakness', 'throat redness', 'joint swelling', 'tongue pain', 'redness in or around nose', 'wrinkles on skin', 'foot or toe weakness', 'hand or finger cramps or spasms', 'back stiffness or tightness', 'wrist lump or mass', 'skin pain', 'low back stiffness or tightness', 'low urine output', 'skin on head or neck looks infected', 'stuttering or stammering', 'problems with orgasm', 'nose deformity', 'lump over jaw', 'sore in nose', 'hip weakness', 'back swelling', 'ankle stiffness or tightness', 'ankle weakness', 'neck weakness', 'num_symptoms']

Data types:

diseases	object
anxiety and nervousness	int64
depression	int64
shortness of breath	int64
depressive or psychotic symptoms	int64
...	...
back swelling	int64
ankle stiffness or tightness	int64
ankle weakness	int64
neck weakness	int64
num_symptoms	int64

Length: 379, dtype: object

Label column: diseases

Total missing values in dataset: 0

Columns with missing values:
Series([], dtype: int64)

Number of duplicate rows: 57298

Number of unique diseases: 773

Top 10 most common diseases:

diseases	
cystitis	1219
nose disorder	1218
vulvodynia	1218
complex regional pain syndrome	1217
spondylosis	1216
vaginal cyst	1215
esophagitis	1215
peripheral nerve disorder	1215
hypoglycemia	1215
conjunctivitis due to allergy	1215

Name: count, dtype: int64

Top 10 least common diseases:

diseases	
diabetes	1
thalassemia	1
heat stroke	1
gas gangrene	1
typhoid fever	1
open wound of the head	1
myocarditis	1
chronic ulcer	1
hypergammaglobulinemia	1
kaposi sarcoma	1

Name: count, dtype: int64

Summary of class counts:

count	773.000000
mean	319.463131
std	351.691549
min	1.000000
25%	30.000000
50%	168.000000
75%	505.000000
max	1219.000000

Name: count, dtype: float64

Number of feature columns: 378

Feature unique-value summary:

count	378.000000
mean	1.896825

```
std      0.620123
min      1.000000
25%      2.000000
50%      2.000000
75%      2.000000
max      12.000000
dtype: float64
```

Number of non-binary feature columns: 1
Some non-binary columns:
['num_symptoms']

Top 20 most frequent symptoms:

num_symptoms	1316921
sharp abdominal pain	32307
vomiting	27874
headache	24719
cough	24296
sharp chest pain	24016
nausea	23687
back pain	21809
shortness of breath	21346
fever	20394
dizziness	17272
abnormal appearing skin	16425
nasal congestion	16200
leg pain	16189
skin swelling	15165
depressive or psychotic symptoms	15064
lower abdominal pain	14936
sore throat	14005
burning abdominal pain	12981
skin rash	12375

dtype: int64

Top 20 least frequent symptoms:

hip swelling	0
foot or toe cramps or spasms	0
pus in sputum	0
low back swelling	0
lump over jaw	0
nose deformity	0
throat irritation	0
mass on ear	0
elbow stiffness or tightness	0
feet turned in	0
hip lump or mass	0
hip weakness	0
discharge in stools	0
disturbance of smell or taste	0
swollen tongue	0
neck weakness	0
ankle stiffness or tightness	0
back swelling	0
pupils unequal	0
infrequent menstruation	0

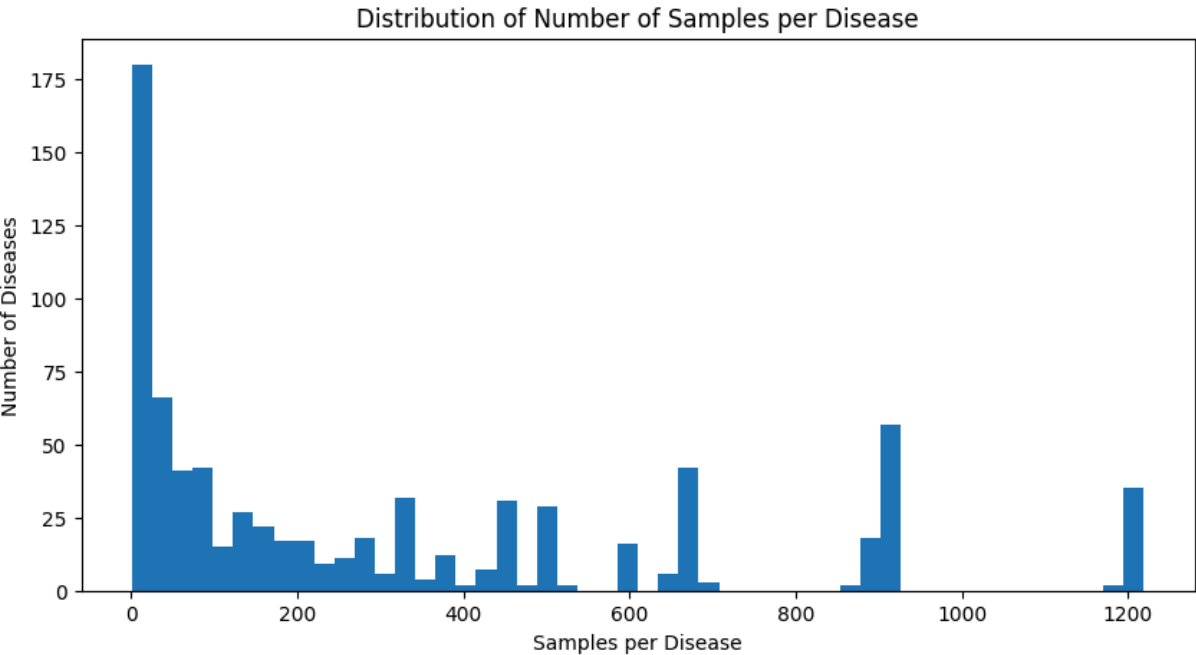
dtype: int64

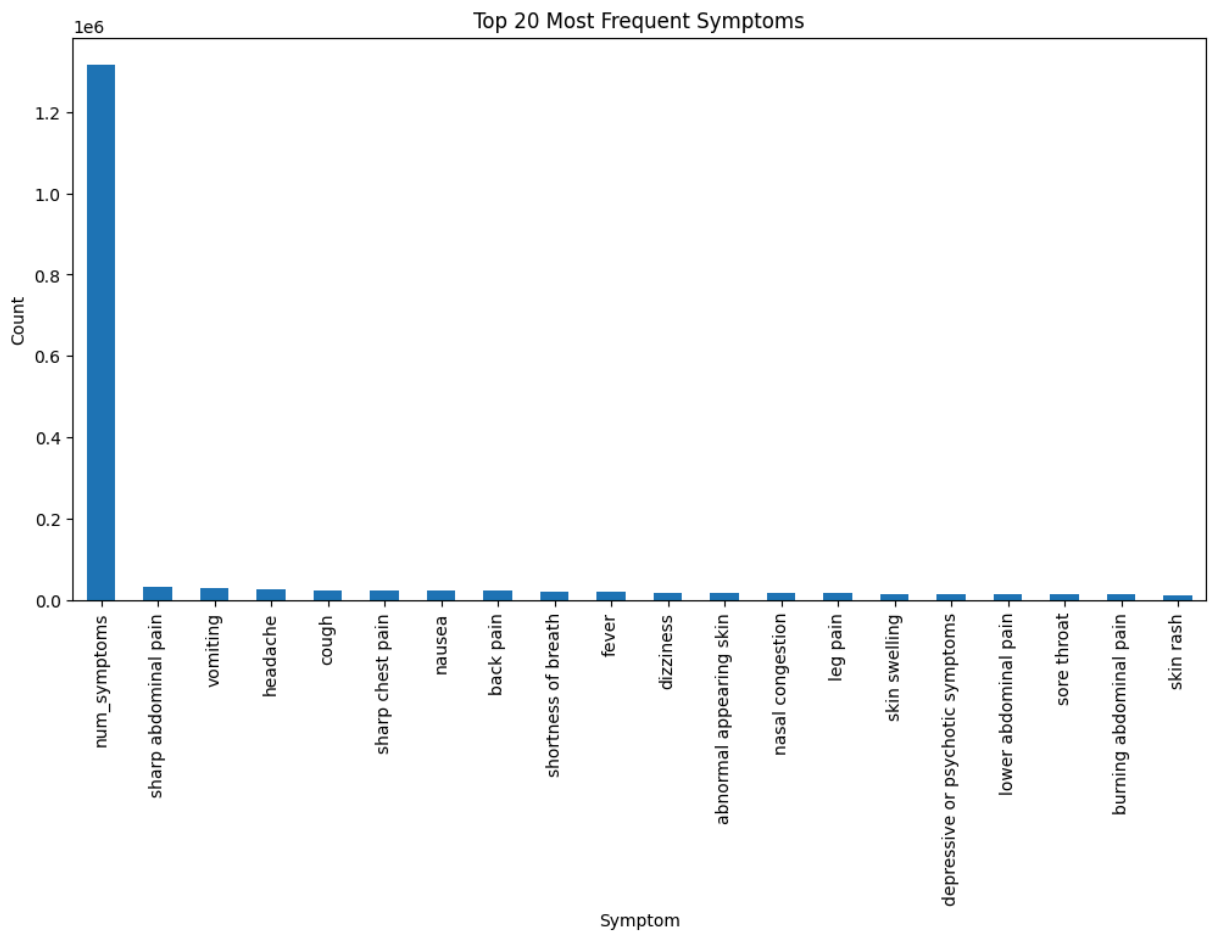
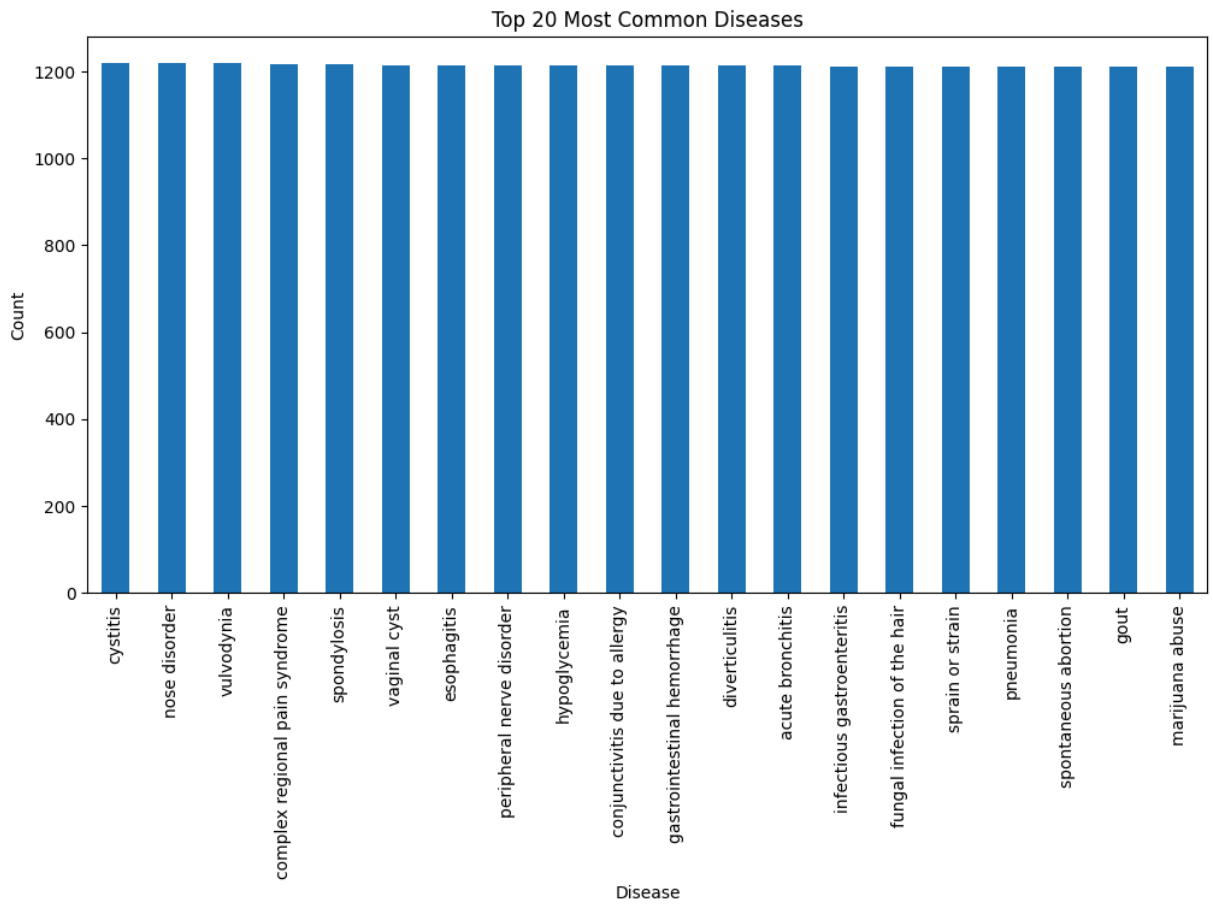
Number of diseases with fewer than 2 samples: 96

Diseases with fewer than 2 samples:

diseases	
acute fatty liver of pregnancy (aflp)	9
valley fever	9
placenta previa	9
hemarthrosis	9
insulin overdose	9
..	
open wound of the head	1
myocarditis	1
chronic ulcer	1
hypergammaglobulinemia	1
kaposi sarcoma	1

Name: count, Length: 96, dtype: int64





Final EDA Summary

Rows: 246945
Columns: 379
Features: 378
Unique diseases: 773
Total missing values: 0
Duplicate rows: 57298
Diseases with <2 samples: 96
Non-binary feature columns: 1

```
In [ ]: # Drop rows where label is missing
df_clean = df.dropna(subset=[label_col]).copy()

# Fill missing feature values with 0 (symptom absent)
df_clean = df_clean.fillna(0)

# Remove duplicates
df_clean = df_clean.drop_duplicates()

print("Final shape:", df_clean.shape)
```

Final shape: (189647, 379)

```
In [ ]: from sklearn.preprocessing import LabelEncoder
```

```
In [ ]: from sklearn.preprocessing import LabelEncoder
y = df_clean[label_col]
class_counts = y.value_counts()
# Remove rare classes from dataset
valid_classes = class_counts[class_counts >= 10].index

df_clean = df_clean[df_clean[label_col].isin(valid_classes)].copy()

df_clean = df_clean.sample(n=100000, random_state=42)
```

```
In [ ]: from sklearn.feature_selection import mutual_info_classif

# redo X, y after filtering
X = df_clean.drop(columns=[label_col])
y = df_clean[label_col]
y_encoded = LabelEncoder().fit_transform(y)
# Calculat mutual info scores
mi_scores = mutual_info_classif(X, y_encoded)
ascending_scores = np.argsort(mi_scores)
features_scores = pd.Series(mi_scores, index=X.columns)
features_scores = features_scores.sort_values(ascending=False)
print(features_scores)
```

```
In [ ]: # run this, so mutual information doesn't have to run again.
selected_feature_cols = ['vomiting', 'sharp abdominal pain', 'headache', 'na
```

```
In [ ]: # selected_feature_cols = features_scores[features_scores > 0.01].index.tolist()
# Ensure the label column is also kept
all_cols_to_keep = [label_col] + selected_feature_cols
```

```
df_clean = df_clean[all_cols_to_keep]
```

In []:

```
In [ ]: # Running mutual information takes a while, here is a cut down list of the t  
print(selected_feature_cols)
```

['vomiting', 'sharp abdominal pain', 'headache', 'nausea', 'sharp chest pain', 'cough', 'back pain', 'fever', 'shortness of breath', 'leg pain', 'abnormal appearing skin', 'nasal congestion', 'dizziness', 'lower abdominal pain', 'skin swelling', 'depressive or psychotic symptoms', 'burning abdominal pain', 'sore throat', 'weakness', 'skin lesion', 'arm pain', 'low back pain', 'skin rash', 'side pain', 'depression', 'loss of sensation', 'diarrhea', 'neck pain', 'ear pain', 'itching of skin', 'abnormal involuntary movements', 'insomnia', 'pelvic pain', 'hip pain', 'skin growth', 'pain in eye', 'chest tightness', 'foot or toe pain', 'problems with movement', 'knee pain', 'peripheral edema', 'coryza', 'shoulder pain', 'facial pain', 'difficulty breathing', 'skin dryness, peeling, scaliness, or roughness', 'painful urination', 'acne or pimples', 'hostile behavior', 'anxiety and nervousness', 'symptoms of eye', 'diminished vision', 'heartburn', 'retention of urine', 'irregular appearing scalp', 'suprapubic pain', 'lower body pain', 'excessive anger', 'blood in stool', 'paresthesia', 'lacrimation', 'temper problems', 'ache all over', 'decreased appetite', 'skin moles', 'frequent urination', 'itchiness of eye', 'fainting', 'upper abdominal pain', 'chills', 'hand or finger pain', 'leg swelling', 'blood in urine', 'abusing alcohol', 'delusions or hallucinations', 'eye redness', 'vaginal discharge', 'allergic reaction', 'involuntary urination', 'num_symptoms', 'cramps and spasms', 'palpitations', 'pain during pregnancy', 'difficulty speaking', 'wrist pain', 'irregular heartbeats', 'elbow pain', 'intermenstrual bleeding', 'drug abuse', 'swollen eye', 'difficulty in swallowing', 'fears and phobias', 'disturbance of memory', 'lip swelling', 'diminished hearing', 'rectal bleeding', 'wheezing', 'foreign body sensation in eye', 'spots or clouds in vision', 'joint pain', 'ankle pain', 'skin irritation', 'hand or finger swelling', 'fluid retention', 'plugged feeling in ear', 'fatigue', 'double vision', 'problems during pregnancy', 'sweating', 'wrist swelling', 'knee swelling', 'vaginal itching', 'regurgitation.1', 'foot or toe swelling', 'vomiting blood', 'heavy menstrual flow', 'pulling at ears', 'feeling ill', 'hurts to breathe', 'bones are painful', 'symptoms of bladder', 'eye burns or stings', 'seizures', 'constipation', 'pain of the anus', 'vaginal pain', 'arm swelling', 'coughing up sputum', 'blood clots during menstrual periods', 'low self-esteem', 'increased heart rate', 'fluid in ear', 'hoarse voice', 'redness in ear', 'white discharge from eye', 'warts', 'lack of growth', 'mouth ulcer', 'focal weakness', 'leg weakness', 'sleepiness', 'spotting or bleeding during pregnancy', 'arm stiffness or tightness', 'toothache', 'restlessness', 'jaw swelling', 'abnormal breathing sounds', 'ringing in ear', 'pain in testicles', 'groin pain', 'symptoms of the face', 'elbow swelling', 'changes in stool appearance', 'long menstrual periods', 'painful menstruation', 'unpredictable menstruation', 'smoking problems', 'ankle swelling', 'gum pain', 'decreased heart rate', 'symptoms of the scrotum and testes', 'sinus congestion', 'weight gain', 'rib pain', 'lymph edema', 'mouth pain', 'painful sinuses', 'stomach bloating', 'regurgitation', 'neck swelling', 'throat feels tight', 'leg cramps or spasms', 'hemoptysis', 'throat swelling', 'impotence', 'back mass or lump', 'pain in gums', 'irritable infant', 'apnea', 'slurring words', 'melena', 'neck mass', 'itchy scalp', 'arm weakness', 'eyelid swelling', 'obsessions and compulsions', 'congestion in chest', 'antisocial behavior', 'shoulder stiffness or tightness', 'mass on eyelid', 'hand or finger stiffness or tightness', 'irregular appearing nails', 'abnormal movement of eyelid', 'muscle pain', 'blindness', 'sneezing', 'swollen lymph nodes', 'pain during intercourse', 'vaginal redness', 'frontal headache', 'back cramps or spasms', 'itching of the anus', 'hysterical behavior', 'groin mass', 'symptoms of prostate', 'nosebleed', 'frequent menstruation', 'uterine contractions', 'too little hair', 'itchy ear(s)', 'symptoms of the kidneys', 'jaundice', 'excessive urination at night', 'knee lump or mass', 'hesitancy', 'knee weakness', 'flu-like syndrome', 'hand or fi

nger lump or mass', 'swelling of scrotum', 'hot flashes', 'mouth dryness', 'unusual color or odor to urine', 'tongue lesions', 'loss of sex drive', 'bleeding from eye', 'leg stiffness or tightness', 'arm lump or mass', 'bleeding from ear', 'burning chest pain', 'kidney mass', 'infertility', 'recent pregnancy', 'infant feeding problem', 'mass in scrotum', 'diaper rash', 'eyelid lesion or rash', 'hand or finger weakness', 'skin on leg or foot looks infected', 'low urine output', 'muscle cramps, contractures, or spasms', 'cloudy eye', 'knee stiffness or tightness', 'flatulence', 'swollen or red tonsils', 'pain or soreness of breast', 'hip stiffness or tightness', 'dry or flaky scalp', 'nightmares', 'neck stiffness or tightness', 'breathing fast', 'swollen abdomen', 'pus draining from ear', 'dry lips', 'penis pain', 'bumps on penis', 'shoulder lump or mass', 'skin on arm or hand looks infected', 'lump or mass of breast', 'poor circulation', 'stiffness all over', 'bleeding gums', 'penis redness', 'mass or swelling around the anus', 'lump in throat', 'early or late onset of menopause']

```
In [ ]: # Remove classes that appear only once in the sample
sample_counts = df_clean[label_col].value_counts()
valid_sample_classes = sample_counts[sample_counts >= 5].index
df_clean = df_clean[df_clean[label_col].isin(valid_sample_classes)].copy()
```

```
In [ ]: df_clean.shape
```

```
Out[ ]: (99968, 266)
```

```
In [ ]: # redo X, y
X_f = df_clean.drop(columns=[label_col])
le = LabelEncoder()

y_f = df_clean[label_col]
#y_f = LabelEncoder().fit_transform(y_f)
```

```
In [ ]: from sklearn.model_selection import train_test_split

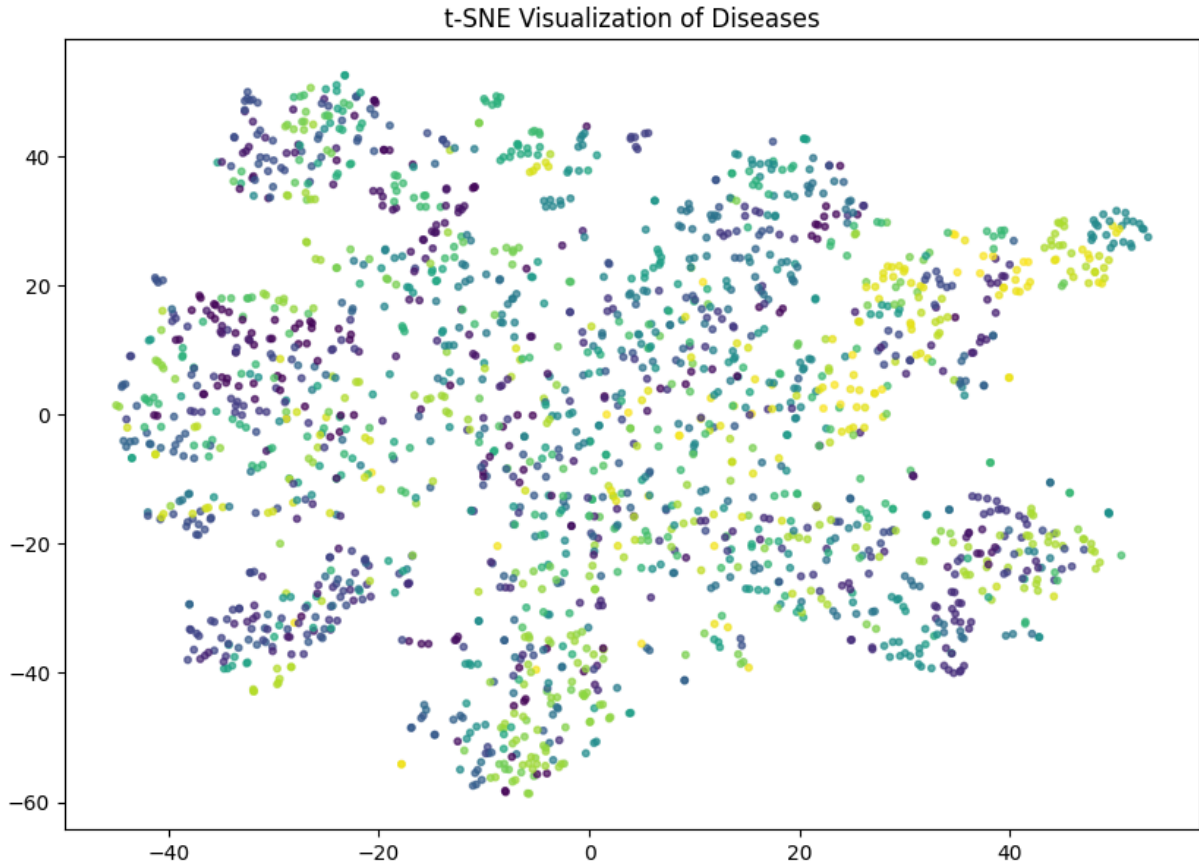
X_train, X_test, y_train, y_test = train_test_split(
    X_f, y_f,
    test_size=0.2,
    random_state=42,
    stratify=y_f
)
```

```
In [ ]: from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

# optional: sample if dataset is large
max_samples = 2000
if len(X) > max_samples:
    idx = np.random.choice(len(X), max_samples, replace=False)
    X_tsne_input = X.iloc[idx]
    y_tsne_input = y_encoded[idx]
else:
    X_tsne_input = X
    y_tsne_input = y_encoded
```

```
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X_tsne_input)

plt.figure(figsize=(10,7))
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y_tsne_input, s=10, alpha=0.7)
plt.title("t-SNE Visualization of Diseases")
plt.show()
```



```
In [ ]: print(y_f[1:5])
```

```
[454 134 529 113]
```

```
In [ ]: print(len(y))
```

```
189647
```

```
In [ ]: from sklearn.dummy import DummyClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    confusion_matrix
)

# -----
# 5-fold stratified cross-validation
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```

# store fold-level metrics for downstream stats / ANOVA
fold_results_dummy = []

# to accumulate predictions across all folds for one overall confusion matrix
all_y_true = []
all_y_pred = []

for fold, (train_idx, test_idx) in enumerate(cv.split(X, y), start=1):
    X_train_fold = X.iloc[train_idx]
    X_test_fold = X.iloc[test_idx]
    y_train_fold = y.iloc[train_idx]
    y_test_fold = y.iloc[test_idx]

    # model
    dummy = DummyClassifier(strategy="most_frequent")
    dummy.fit(X_train_fold, y_train_fold)
    y_pred_fold = dummy.predict(X_test_fold)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_divi
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="mac
    macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", z

    fold_results_dummy.append({
        "model": "DummyClassifier_most_frequent",
        "fold": fold,
        "accuracy": acc,
        "macro_f1": macro_f1,
        "macro_precision": macro_precision,
        "macro_recall": macro_recall
    })

    all_y_true.extend(y_test_fold)
    all_y_pred.extend(y_pred_fold)

    print(f"\nFold {fold}")
    print(f"Accuracy: {acc:.4f}")
    print(f"Macro F1: {macro_f1:.4f}")
    print(f"Macro Precision: {macro_precision:.4f}")
    print(f"Macro Recall: {macro_recall:.4f}")

# -----
# convert fold metrics to dataframe
# -----
results_df = pd.DataFrame(fold_results_dummy)

print("\n=== Fold-level results ===")
print(results_df)

# save for ANOVA / downstream analysis
results_df.to_csv("dummy_cv_fold_results.csv", index=False)

# -----
# summary statistics

```

```

# -----
n = 5
acc_mean = results_df["accuracy"].mean()
print(acc_mean)
acc_std = results_df["accuracy"].std()
acc_se = acc_std / np.sqrt(n)

f1_mean = results_df["macro_f1"].mean()
print(f1_mean)
f1_std = results_df["macro_f1"].std()
f1_se = f1_std / np.sqrt(n)

model_label = ['DummyClassifier_Frequent']
supervised_mean_accuracy = [acc_mean]
supervised_mean_accuracy_se = [acc_se]
supervised_mean_f1 = [f1_mean]
supervised_mean_f1_se = [f1_se]

```

```

In [ ]: from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.metrics import confusion_matrix
# Due to logistic regression being computationally expensive due to the large dataset

svc = LinearSVC(
    max_iter=2000
)

# -----
# 5-fold stratified cross-validation
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# store fold-level metrics for downstream stats / ANOVA
fold_results_svc = []

# to accumulate predictions across all folds for one overall confusion matrix
all_y_true = []
all_y_pred = []

for fold, (train_idx, test_idx) in enumerate(cv.split(X_f, y_f), start=1):
    X_train_fold = X_f.iloc[train_idx]
    X_test_fold = X_f.iloc[test_idx]
    y_train_fold = y_f.iloc[train_idx]
    y_test_fold = y_f.iloc[test_idx]

    # model
    svc = LinearSVC(max_iter=2000)
    svc.fit(X_train_fold, y_train_fold)
    y_pred_fold = svc.predict(X_test_fold)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_division=0)
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="macro", zero_division=0)
    macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", zero_division=0)

```

```

fold_results_svc.append({
    "model": "Linear SVC",
    "fold": fold,
    "accuracy": acc,
    "macro_f1": macro_f1,
    "macro_precision": macro_precision,
    "macro_recall": macro_recall
})

all_y_true.extend(y_test_fold)
all_y_pred.extend(y_pred_fold)

print(f"\nFold {fold}")
print(f"Accuracy: {acc:.4f}")
print(f"Macro F1: {macro_f1:.4f}")
print(f"Macro Precision: {macro_precision:.4f}")
print(f"Macro Recall: {macro_recall:.4f}")

# -----
# convert fold metrics to dataframe
# -----
results_df2 = pd.DataFrame(fold_results_svc)
# save for ANOVA / downstream analysis
results_df2.to_csv("svc_cv_fold_results.csv", index=False)

print("\n=== Fold-level results ===")
print(results_df2)

b
# -----
# summary statistics
# -----
n = 5
acc_mean_svc = results_df2["accuracy"].mean()
print(acc_mean_svc)
acc_std_svc = results_df2["accuracy"].std()
acc_se_svc = acc_std_svc / np.sqrt(n)

f1_mean_svc = results_df2["macro_f1"].mean()
print(f1_mean_svc)
f1_std_svc = results_df2["macro_f1"].std()
f1_se_svc = f1_std_svc / np.sqrt(n)

model_label.append('Linear SVC')
supervised_mean_accuracy.append(acc_mean_svc)
supervised_mean_accuracy_se.append(acc_se_svc)
supervised_mean_f1.append(f1_mean_svc)
supervised_mean_f1_se.append(f1_se_svc)

```


Fold 1
Accuracy: 0.8328
Macro F1: 0.7796
Macro Precision: 0.8049
Macro Recall: 0.7810

Fold 2
Accuracy: 0.8298
Macro F1: 0.7761
Macro Precision: 0.7941
Macro Recall: 0.7794

Fold 3
Accuracy: 0.8277
Macro F1: 0.7816
Macro Precision: 0.7969
Macro Recall: 0.7858

Fold 4
Accuracy: 0.8290
Macro F1: 0.7846
Macro Precision: 0.8058
Macro Recall: 0.7881

Fold 5
Accuracy: 0.8263
Macro F1: 0.7712
Macro Precision: 0.7960
Macro Recall: 0.7742

=== Fold-level results ===

	model	fold	accuracy	macro_f1	macro_precision	macro_recall
0	Linear SVC	1	0.832850	0.779551	0.804853	0.781013
1	Linear SVC	2	0.829799	0.776101	0.794124	0.779367
2	Linear SVC	3	0.827748	0.781606	0.796881	0.785830
3	Linear SVC	4	0.828990	0.784624	0.805751	0.788054
4	Linear SVC	5	0.826289	0.771153	0.796018	0.774162

0.8291352933814922
0.7786069600807097

```
In [ ]: n = 5
acc_mean_svc = results_df2["accuracy"].mean()
print(acc_mean_svc)
acc_std_svc = results_df2["accuracy"].std()
acc_se_svc = acc_std_svc / np.sqrt(n)

f1_mean_svc = results_df2["macro_f1"].mean()
print(f1_mean_svc)
f1_std_svc = results_df2["macro_f1"].std()
f1_se_svc = f1_std_svc / np.sqrt(n)

model_label.append('Linear SVC')
supervised_mean_accuracy.append(acc_mean_svc)
supervised_mean_accuracy_se.append(acc_se_svc)
supervised_mean_f1.append(f1_mean_svc)
supervised_mean_f1_se.append(f1_se_svc)
```

```
0.8291352933814922
0.7786069600807097
```

```
In [ ]: del model_label[1]
```

```
In [ ]: print(model_label)
print(supervised_mean_accuracy)
print(supervised_mean_accuracy_se)
print(supervised_mean_f1)
print(supervised_mean_f1_se)
```

```
['DummyClassifier_Frequent', 'Linear SVC']
[np.float64(0.010753441607933789), np.float64(0.8291352933814922)]
[np.float64(0.00027349477499808935), np.float64(0.0011018934835920482)]
[np.float64(5.613037211959273e-05), np.float64(0.7786069600807097)]
[np.float64(1.762846862092231e-06), np.float64(0.0023230624420774175)]
```

```
In [ ]: import tensorflow as tf
# -----
# Optional but recommended for reproducibility
# -----
np.random.seed(40)
tf.random.set_seed(40)

le = LabelEncoder()
y_f = pd.Series(le.fit_transform(y_f), index=y_f.index)
# -----
# helper function to build the MLP
# -----
def build_mlp(input_dim, num_classes):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),

        layers.Dense(256, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(128, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# -----
# 5-fold stratified cross-validation
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```

# store fold-level metrics
fold_results_mlp = []

# to accumulate predictions across all folds for one overall confusion matrix
all_y_true_mlp = []
all_y_pred_mlp = []

# if X_f is a pandas DataFrame and y_f is a pandas Series, this works directly
input_dim = X_f.shape[1]
num_classes = len(np.unique(y_f))

for fold, (train_idx, test_idx) in enumerate(cv.split(X_f, y_f), start=1):
    # split data
    X_train_fold = X_f.iloc[train_idx]
    X_test_fold = X_f.iloc[test_idx]
    y_train_fold = y_f.iloc[train_idx]
    y_test_fold = y_f.iloc[test_idx]

    # convert to numpy arrays for keras
    X_train_fold = X_train_fold.to_numpy().astype("float32")
    X_test_fold = X_test_fold.to_numpy().astype("float32")
    y_train_fold = y_train_fold.to_numpy()
    y_test_fold = y_test_fold.to_numpy()

    # clear previous keras graph/session
    keras.backend.clear_session()

    # build fresh model for this fold
    model = build_mlp(input_dim=input_dim, num_classes=num_classes)

    # early stopping
    early_stopping = keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True
    )

    # train model
    history = model.fit(
        X_train_fold, y_train_fold,
        validation_split=0.2,
        epochs=100,
        batch_size=256,
        callbacks=[early_stopping],
        verbose=0
    )

    # predict
    y_pred_proba_fold = model.predict(X_test_fold, verbose=0)
    y_pred_fold = np.argmax(y_pred_proba_fold, axis=1)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_divi
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="mac

```

```

macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", z

fold_results_mlp.append({
    "model": "MLP",
    "fold": fold,
    "accuracy": acc,
    "macro_f1": macro_f1,
    "macro_precision": macro_precision,
    "macro_recall": macro_recall
})

all_y_true_mlp.extend(y_test_fold)
all_y_pred_mlp.extend(y_pred_fold)

print(f"\nFold {fold}")
print(f"Accuracy: {acc:.4f}")
print(f"Macro F1: {macro_f1:.4f}")
print(f"Macro Precision: {macro_precision:.4f}")
print(f"Macro Recall: {macro_recall:.4f}")

# -----
# convert fold metrics to dataframe
# -----
results_df_mlp = pd.DataFrame(fold_results_mlp)
# save for ANOVA / downstream analysis
results_df_mlp.to_csv("mlp_cv_fold_results.csv", index=False)

print("\n=== Fold-level results ===")
print(results_df_mlp)

# -----
# summary statistics
# -----
n = 5
acc_mean_mlp = results_df_mlp["accuracy"].mean()
print(acc_mean_mlp)
acc_std_mlp = results_df_mlp["accuracy"].std()
acc_se_mlp = acc_std_mlp / np.sqrt(n)

f1_mean_mlp = results_df_mlp["macro_f1"].mean()
print(f1_mean_mlp)
f1_std_mlp = results_df_mlp["macro_f1"].std()
f1_se_mlp = f1_std_mlp / np.sqrt(n)

model_label.append('Multi-Layer Perceptron')
supervised_mean_accuracy.append(acc_mean_mlp)
supervised_mean_accuracy_se.append(acc_se_mlp)
supervised_mean_f1.append(f1_mean_mlp)
supervised_mean_f1_se.append(f1_se_mlp)

```

Fold 1
Accuracy: 0.8245
Macro F1: 0.7559
Macro Precision: 0.7934
Macro Recall: 0.7513

Fold 2
Accuracy: 0.8258
Macro F1: 0.7617
Macro Precision: 0.7906
Macro Recall: 0.7605

Fold 3
Accuracy: 0.8230
Macro F1: 0.7660
Macro Precision: 0.8011
Macro Recall: 0.7633

Fold 4
Accuracy: 0.8212
Macro F1: 0.7649
Macro Precision: 0.8003
Macro Recall: 0.7640

Fold 5
Accuracy: 0.8183
Macro F1: 0.7531
Macro Precision: 0.7934
Macro Recall: 0.7480

=== Fold-level results ===

	model	fold	accuracy	macro_f1	macro_precision	macro_recall
0	MLP	1	0.824547	0.755898	0.793407	0.751346
1	MLP	2	0.825848	0.761688	0.790611	0.760480
2	MLP	3	0.822997	0.766032	0.801117	0.763283
3	MLP	4	0.821237	0.764876	0.800294	0.764044
4	MLP	5	0.818336	0.753115	0.793429	0.748038
			0.8225931736906157			
			0.7603215373762231			

```
In [ ]: from tensorflow import keras
        from tensorflow.keras import layers
        import numpy as np

        model = keras.Sequential([
            layers.Input(shape=(X_f.shape[1],)), # Use X_f.shape[1] for the correct
            layers.Dense(256, activation='relu'),
            layers.BatchNormalization(),
            layers.Dropout(0.3),

            layers.Dense(128, activation='relu'),
            layers.BatchNormalization(),
            layers.Dropout(0.3),

            layers.Dense(len(np.unique(y_f)), activation='softmax')])
```

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 256)	68
batch_normalization (BatchNormalization)	(None, 256)	1
dropout (Dropout)	(None, 256)	
dense_1 (Dense)	(None, 128)	32
batch_normalization_1 (BatchNormalization)	(None, 128)	
dropout_1 (Dropout)	(None, 128)	
dense_2 (Dense)	(None, 579)	74

Total params: 177,219 (692.26 KB)

Trainable params: 176,451 (689.26 KB)

Non-trainable params: 768 (3.00 KB)

```

In [ ]: early_stopping = keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stopping]
)

plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='val')
plt.legend()
plt.title('Loss of MLP')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.show()

y_pred_proba = model.predict(X_test)
y_pred = np.argmax(y_pred_proba, axis=1)

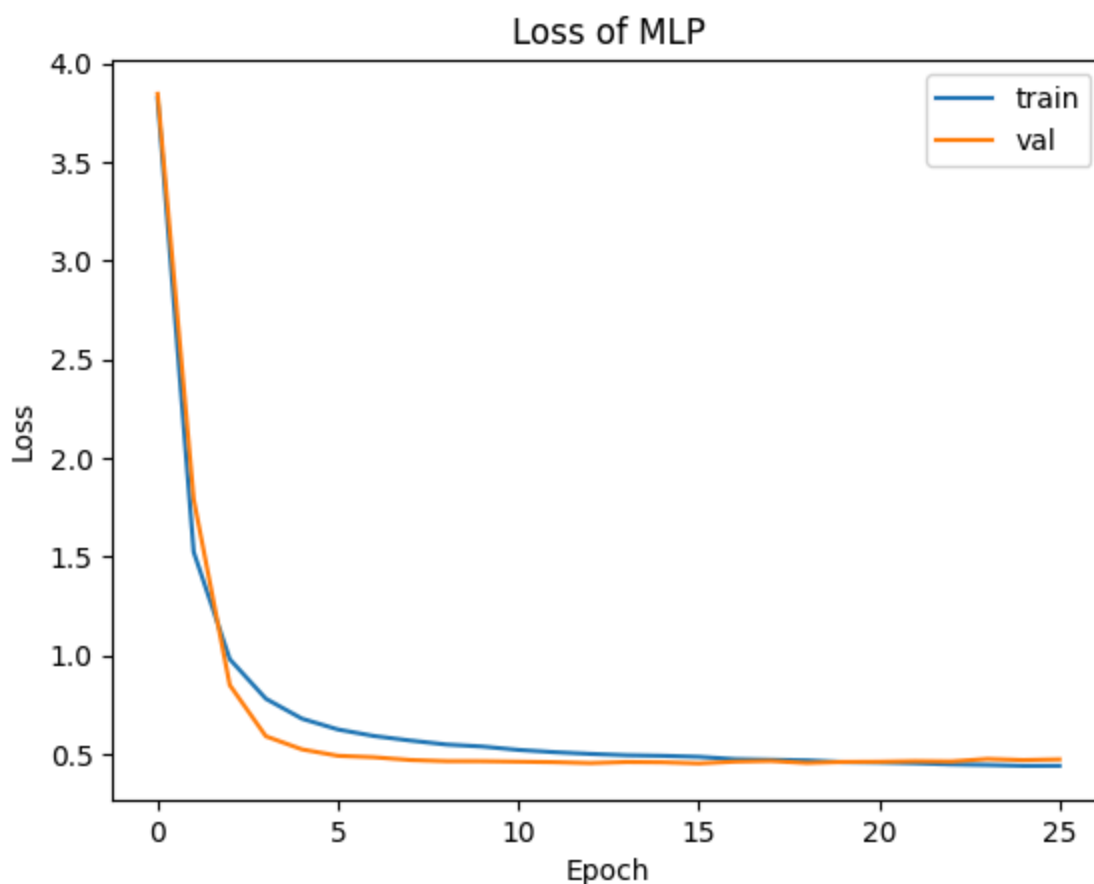
```

```
print("\n=== Multilayer Perceptron (Neural Network) ===")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Macro F1:", f1_score(y_test, y_pred, average="macro"))
print('Precision:', precision_score(y_test, y_pred, average='macro'))
print('Recall:', recall_score(y_test, y_pred, average='macro'))

# plot and display confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(20,20))
sns.heatmap(cm, annot=False)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Epoch 1/100
153/153  **6s** 19ms/step - accuracy: 0.3576 - loss: 3.8226
- val_accuracy: 0.6151 - val_loss: 3.8464
Epoch 2/100
153/153  **3s** 17ms/step - accuracy: 0.6668 - loss: 1.5228
- val_accuracy: 0.7631 - val_loss: 1.7902
Epoch 3/100
153/153  **3s** 17ms/step - accuracy: 0.7518 - loss: 0.9790
- val_accuracy: 0.8003 - val_loss: 0.8478
Epoch 4/100
153/153  **3s** 20ms/step - accuracy: 0.7795 - loss: 0.7786
- val_accuracy: 0.8144 - val_loss: 0.5883
Epoch 5/100
153/153  **4s** 26ms/step - accuracy: 0.7946 - loss: 0.6775
- val_accuracy: 0.8175 - val_loss: 0.5217
Epoch 6/100
153/153  **3s** 16ms/step - accuracy: 0.8067 - loss: 0.6226
- val_accuracy: 0.8209 - val_loss: 0.4891
Epoch 7/100
153/153  **2s** 16ms/step - accuracy: 0.8108 - loss: 0.5895
- val_accuracy: 0.8212 - val_loss: 0.4818
Epoch 8/100
153/153  **3s** 16ms/step - accuracy: 0.8181 - loss: 0.5666
- val_accuracy: 0.8247 - val_loss: 0.4679
Epoch 9/100
153/153  **3s** 19ms/step - accuracy: 0.8196 - loss: 0.5464
- val_accuracy: 0.8214 - val_loss: 0.4619
Epoch 10/100
153/153  **5s** 30ms/step - accuracy: 0.8213 - loss: 0.5362
- val_accuracy: 0.8240 - val_loss: 0.4616
Epoch 11/100
153/153  **2s** 16ms/step - accuracy: 0.8234 - loss: 0.5190
- val_accuracy: 0.8236 - val_loss: 0.4593
Epoch 12/100
153/153  **3s** 16ms/step - accuracy: 0.8273 - loss: 0.5076
- val_accuracy: 0.8258 - val_loss: 0.4569
Epoch 13/100
153/153  **3s** 16ms/step - accuracy: 0.8293 - loss: 0.4988
- val_accuracy: 0.8259 - val_loss: 0.4521
Epoch 14/100
153/153  **3s** 19ms/step - accuracy: 0.8311 - loss: 0.4922
- val_accuracy: 0.8218 - val_loss: 0.4576
Epoch 15/100
153/153  **4s** 27ms/step - accuracy: 0.8303 - loss: 0.4891
- val_accuracy: 0.8220 - val_loss: 0.4562
Epoch 16/100
153/153  **3s** 17ms/step - accuracy: 0.8312 - loss: 0.4839
- val_accuracy: 0.8270 - val_loss: 0.4504
Epoch 17/100
153/153  **3s** 16ms/step - accuracy: 0.8352 - loss: 0.4723
- val_accuracy: 0.8259 - val_loss: 0.4596
Epoch 18/100
153/153  **4s** 26ms/step - accuracy: 0.8380 - loss: 0.4692
- val_accuracy: 0.8246 - val_loss: 0.4629
Epoch 19/100
153/153  **4s** 24ms/step - accuracy: 0.8383 - loss: 0.4660

- val_accuracy: 0.8284 - val_loss: 0.4527
Epoch 20/100
153/153 ————— **3s** 21ms/step - accuracy: 0.8404 - loss: 0.4572
- val_accuracy: 0.8267 - val_loss: 0.4581
Epoch 21/100
153/153 ————— **2s** 16ms/step - accuracy: 0.8390 - loss: 0.4552
- val_accuracy: 0.8268 - val_loss: 0.4590
Epoch 22/100
153/153 ————— **3s** 16ms/step - accuracy: 0.8428 - loss: 0.4513
- val_accuracy: 0.8260 - val_loss: 0.4620
Epoch 23/100
153/153 ————— **3s** 16ms/step - accuracy: 0.8434 - loss: 0.4457
- val_accuracy: 0.8243 - val_loss: 0.4604
Epoch 24/100
153/153 ————— **3s** 22ms/step - accuracy: 0.8434 - loss: 0.4427
- val_accuracy: 0.8268 - val_loss: 0.4732
Epoch 25/100
153/153 ————— **4s** 16ms/step - accuracy: 0.8446 - loss: 0.4382
- val_accuracy: 0.8266 - val_loss: 0.4675
Epoch 26/100
153/153 ————— **2s** 16ms/step - accuracy: 0.8466 - loss: 0.4383
- val_accuracy: 0.8275 - val_loss: 0.4710



32/32 ————— **0s** 5ms/step

=== Multilayer Perceptron (Neural Network) ===

Accuracy: 0.8144433299899699

Macro F1: 0.7616909915062949

Precision: 0.7812246094567995

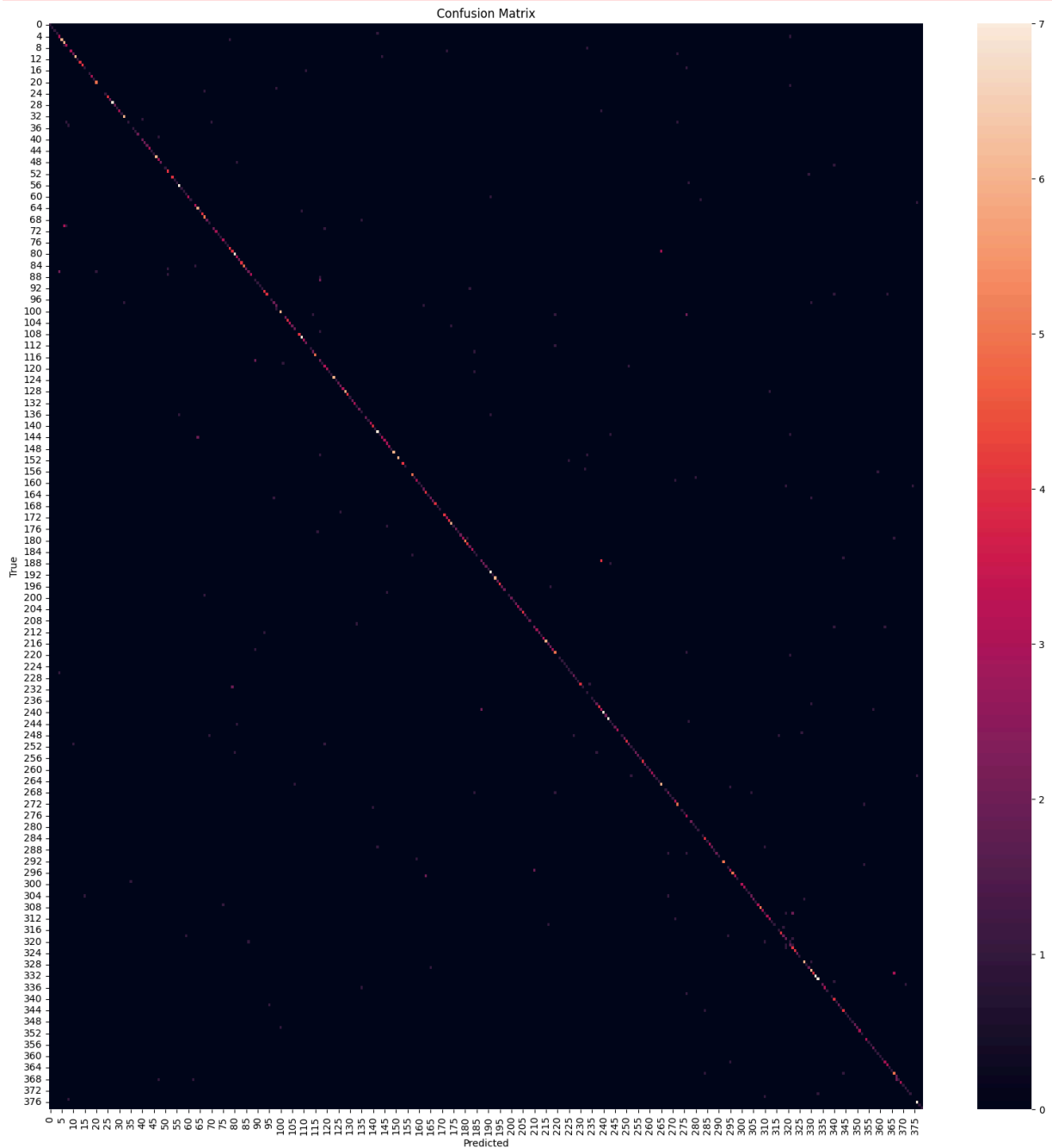
Recall: 0.7777735896469404

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1
565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
n labels with no predicted samples. Use `zero_division` parameter to control
this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1
565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in l
abels with no true samples. Use `zero_division` parameter to control this be
havior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



```
In [ ]: from xgboost import XGBClassifier
        from sklearn.metrics import accuracy_score, f1_score
        import time
        import numpy as np
```

```

# -----
# 5-fold stratified cross-validation
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# store fold-level metrics for downstream stats / ANOVA
fold_results_xgb = []

# to accumulate predictions across all folds for one overall confusion matrix
all_y_true = []
all_y_pred = []

for fold, (train_idx, test_idx) in enumerate(cv.split(X_f, y_f), start=1):
    X_train_fold = X_f.iloc[train_idx]
    X_test_fold = X_f.iloc[test_idx]
    y_train_fold = y_f.iloc[train_idx]
    y_test_fold = y_f.iloc[test_idx]

    # model
    xgb = XGBClassifier(
        objective="multi:softmax",
        num_class=len(np.unique(y_train_fold)),
        n_estimators=40,
        max_depth=3,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_weight=3,
        eval_metric="mlogloss",
        tree_method="hist",
        random_state=42,
        n_jobs=-1
    )
    xgb.fit(X_train_fold, y_train_fold)
    y_pred_fold = xgb.predict(X_test_fold)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_divi
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="mac
    macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", z

    fold_results_xgb.append({
        "model": "XG Boost",
        "fold": fold,
        "accuracy": acc,
        "macro_f1": macro_f1,
        "macro_precision": macro_precision,
        "macro_recall": macro_recall
    })

    all_y_true.extend(y_test_fold)
    all_y_pred.extend(y_pred_fold)

print(f"\nFold {fold}")
print(f"Accuracy: {acc:.4f}")

```

```

    print(f"Macro F1: {macro_f1:.4f}")
    print(f"Macro Precision: {macro_precision:.4f}")
    print(f"Macro Recall: {macro_recall:.4f}")

# -----
# convert fold metrics to dataframe
# -----
results_df3 = pd.DataFrame(fold_results_xgb)
# save for ANOVA / downstream analysis
results_df3.to_csv("xgb_cv_fold_results.csv", index=False)

print("\n=== Fold-level results ===")
print(results_df3)

# -----
# summary statistics
# -----
n = 5
acc_mean_xgb = results_df3["accuracy"].mean()
print(acc_mean_xgb)
acc_std_xgb = results_df3["accuracy"].std()
acc_se_xgb = acc_std_xgb / np.sqrt(n)

f1_mean_xgb = results_df3["macro_f1"].mean()
print(f1_mean_xgb)
f1_std_xgb = results_df3["macro_f1"].std()
f1_se_xgb = f1_std_xgb / np.sqrt(n)

model_label.append('XG Boost')
supervised_mean_accuracy.append(acc_mean_xgb)
supervised_mean_accuracy_se.append(acc_se_xgb)
supervised_mean_f1.append(f1_mean_xgb)
supervised_mean_f1_se.append(f1_se_xgb)

```

Fold 1
Accuracy: 0.7838
Macro F1: 0.4880
Macro Precision: 0.4927
Macro Recall: 0.4915

Fold 2
Accuracy: 0.7788
Macro F1: 0.4883
Macro Precision: 0.4937
Macro Recall: 0.4935

Fold 3
Accuracy: 0.7788
Macro F1: 0.4823
Macro Precision: 0.4859
Macro Recall: 0.4877

Fold 4
Accuracy: 0.7742
Macro F1: 0.4813
Macro Precision: 0.4825
Macro Recall: 0.4875

Fold 5
Accuracy: 0.7770
Macro F1: 0.4841
Macro Precision: 0.4918
Macro Recall: 0.4880

=== Fold-level results ===

	model	fold	accuracy	macro_f1	macro_precision	macro_recall
0	XG Boost	1	0.783785	0.487986	0.492696	0.491470
1	XG Boost	2	0.778784	0.488310	0.493693	0.493469
2	XG Boost	3	0.778784	0.482340	0.485941	0.487710
3	XG Boost	4	0.774221	0.481291	0.482467	0.487499
4	XG Boost	5	0.777022	0.484076	0.491752	0.488027

0.7785190681497951
0.48480062260427187

```
In [ ]: from sklearn.naive_bayes import BernoulliNB
        from sklearn.metrics import accuracy_score, f1_score
        import time

        # -----
        # 5-fold stratified cross-validation
        # -----
        cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

        # store fold-level metrics for downstream stats / ANOVA
        fold_results_nb = []

        # to accumulate predictions across all folds for one overall confusion matrix
        all_y_true = []
        all_y_pred = []
```

```

for fold, (train_idx, test_idx) in enumerate(cv.split(X_f, y_f), start=1):
    X_train_fold = X_f.iloc[train_idx]
    X_test_fold = X_f.iloc[test_idx]
    y_train_fold = y_f.iloc[train_idx]
    y_test_fold = y_f.iloc[test_idx]

    nb = BernoulliNB()
    nb.fit(X_train_fold, y_train_fold)
    y_pred_fold = nb.predict(X_test_fold)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_divi
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="mac
    macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", z

    fold_results_nb.append({
        "model": "Bernoulli Naive Bayes",
        "fold": fold,
        "accuracy": acc,
        "macro_f1": macro_f1,
        "macro_precision": macro_precision,
        "macro_recall": macro_recall
    })

    all_y_true.extend(y_test_fold)
    all_y_pred.extend(y_pred_fold)

    print(f"\nFold {fold}")
    print(f"Accuracy: {acc:.4f}")
    print(f"Macro F1: {macro_f1:.4f}")
    print(f"Macro Precision: {macro_precision:.4f}")
    print(f"Macro Recall: {macro_recall:.4f}")

# -----
# convert fold metrics to dataframe
# -----
results_df4 = pd.DataFrame(fold_results_nb)
# save for ANOVA / downstream analysis
results_df4.to_csv("bernnb_cv_fold_results.csv", index=False)

print("\n=== Fold-level results ===")
print(results_df4)

# -----
# summary statistics
# -----
n = 5
acc_mean_nb = results_df4["accuracy"].mean()
print(acc_mean_nb)
acc_std_nb = results_df4["accuracy"].std()
acc_se_nb = acc_std_nb / np.sqrt(n)

f1_mean_nb = results_df4["macro_f1"].mean()
print(f1_mean_nb)

```

```

f1_std_nb = results_df4["macro_f1"].std()
f1_se_nb = f1_std_nb / np.sqrt(n)

model_label.append('Bernoulli Naives Bayes')
supervised_mean_accuracy.append(acc_mean_nb)
supervised_mean_accuracy_se.append(acc_se_nb)
supervised_mean_f1.append(f1_mean_nb)
supervised_mean_f1_se.append(f1_se_nb)

```

Fold 1

Accuracy: 0.7989
Macro F1: 0.5397
Macro Precision: 0.6109
Macro Recall: 0.5190

Fold 2

Accuracy: 0.7997
Macro F1: 0.5438
Macro Precision: 0.6107
Macro Recall: 0.5232

Fold 3

Accuracy: 0.7962
Macro F1: 0.5387
Macro Precision: 0.6019
Macro Recall: 0.5192

Fold 4

Accuracy: 0.7961
Macro F1: 0.5398
Macro Precision: 0.6002
Macro Recall: 0.5212

Fold 5

Accuracy: 0.7946
Macro F1: 0.5417
Macro Precision: 0.6085
Macro Recall: 0.5216

=== Fold-level results ===

	model	fold	accuracy	macro_f1	macro_precision	\
0	Bernoulli Naive Bayes	1	0.798890	0.539735	0.610869	
1	Bernoulli Naive Bayes	2	0.799740	0.543833	0.610697	
2	Bernoulli Naive Bayes	3	0.796239	0.538679	0.601880	
3	Bernoulli Naive Bayes	4	0.796079	0.539839	0.600165	
4	Bernoulli Naive Bayes	5	0.794578	0.541691	0.608535	
	macro_recall					
0			0.519040			
1			0.523172			
2			0.519154			
3			0.521220			
4			0.521626			
			0.7971050380787221			
			0.5407555281366032			

```

In [ ]: from sklearn.linear_model import SGDClassifier
# -----
# 5-fold stratified cross-validation
# -----
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# store fold-level metrics for downstream stats / ANOVA
fold_results_sgd = []

# to accumulate predictions across all folds for one overall confusion matrix
all_y_true = []
all_y_pred = []

for fold, (train_idx, test_idx) in enumerate(cv.split(X_f, y_f), start=1):
    X_train_fold = X_f.iloc[train_idx]
    X_test_fold = X_f.iloc[test_idx]
    y_train_fold = y_f.iloc[train_idx]
    y_test_fold = y_f.iloc[test_idx]

    sgd = SGDClassifier(
        loss="log_loss",
        max_iter=1000,
        tol=1e-3,
        random_state=42
    )

    sgd.fit(X_train_fold, y_train_fold)
    y_pred_fold = sgd.predict(X_test_fold)

    # metrics for this fold
    acc = accuracy_score(y_test_fold, y_pred_fold)
    macro_f1 = f1_score(y_test_fold, y_pred_fold, average="macro", zero_divi
    macro_precision = precision_score(y_test_fold, y_pred_fold, average="mac
    macro_recall = recall_score(y_test_fold, y_pred_fold, average="macro", z

    fold_results_sgd.append({
        "model": "Stochastic Gradient Descent",
        "fold": fold,
        "accuracy": acc,
        "macro_f1": macro_f1,
        "macro_precision": macro_precision,
        "macro_recall": macro_recall
    })

    all_y_true.extend(y_test_fold)
    all_y_pred.extend(y_pred_fold)

    print(f"\nFold {fold}")
    print(f"Accuracy: {acc:.4f}")
    print(f"Macro F1: {macro_f1:.4f}")
    print(f"Macro Precision: {macro_precision:.4f}")
    print(f"Macro Recall: {macro_recall:.4f}")

# -----

```



```

# convert fold metrics to dataframe
# -----
results_df5 = pd.DataFrame(fold_results_sgd)
# save for ANOVA / downstream analysis
results_df5.to_csv("sgd_cv_fold_results.csv", index=False)

print("\n=== Fold-level results ===")
print(results_df5)

# -----
# summary statistics
# -----
n = 5
acc_mean_sgd = results_df5["accuracy"].mean()
print(acc_mean_sgd)
acc_std_sgd = results_df5["accuracy"].std()
acc_se_sgd = acc_std_sgd / np.sqrt(n)

f1_mean_sgd = results_df5["macro_f1"].mean()
print(f1_mean_sgd)
f1_std_sgd = results_df5["macro_f1"].std()
f1_se_sgd = f1_std_sgd / np.sqrt(n)

model_label.append('Stochastic Gradient Descent')
supervised_mean_accuracy.append(acc_mean_sgd)
supervised_mean_accuracy_se.append(acc_se_sgd)
supervised_mean_f1.append(f1_mean_sgd)
supervised_mean_f1_se.append(f1_se_sgd)

```

Fold 1
Accuracy: 0.7999
Macro F1: 0.5528
Macro Precision: 0.6005
Macro Recall: 0.5407

Fold 2
Accuracy: 0.8014
Macro F1: 0.5606
Macro Precision: 0.6135
Macro Recall: 0.5472

Fold 3
Accuracy: 0.8030
Macro F1: 0.5583
Macro Precision: 0.6115
Macro Recall: 0.5473

Fold 4
Accuracy: 0.7992
Macro F1: 0.5595
Macro Precision: 0.6122
Macro Recall: 0.5473

Fold 5
Accuracy: 0.8003
Macro F1: 0.5565
Macro Precision: 0.6007
Macro Recall: 0.5464

=== Fold-level results ===

	model	fold	accuracy	macro_f1	macro_precision \
0	Stochastic Gradient Descent	1	0.799940	0.552801	0.600504
1	Stochastic Gradient Descent	2	0.801390	0.560599	0.613493
2	Stochastic Gradient Descent	3	0.802991	0.558273	0.611461
3	Stochastic Gradient Descent	4	0.799180	0.559537	0.612179
4	Stochastic Gradient Descent	5	0.800280	0.556534	0.600704

	macro_recall
0	0.540651
1	0.547201
2	0.547323
3	0.547347
4	0.546404
0.8007562214645487	
0.5575488001433438	

```
In [ ]: print(model_label)
        print(supervised_mean_accuracy)
        print(supervised_mean_accuracy_se)
```

```
['DummyClassifier_Frequent', 'Linear SVC', 'Multi-Layer Perceptron', 'XG Boost', 'Bernouli Naives Bayes', 'Stochastic Gradient Descent']
[np.float64(0.010753441607933789), np.float64(0.8291352933814922), np.float64(0.8225931736906157), np.float64(0.7785190681497951), np.float64(0.7971050380787221), np.float64(0.8007562214645487)]
[np.float64(0.00027349477499808935), np.float64(0.0011018934835920482), np.float64(0.0013140044443233152), np.float64(0.0015586273036248036), np.float64(0.0009569921716542808), np.float64(0.0006623556347034351)]
```

```
In [ ]: model_label[0] = 'Majority Class'
```

```
In [ ]: import pandas as pd
import numpy as np
import glob

supervised_mean_accuracy_sd = []
supervised_mean_f1_sd = []

csv_files = [
    "dummy_cv_fold_results.csv",
    "linear_svc_cv_fold_results.csv",
    "mlp_cv_fold_results.csv",
    "bernnb_cv_fold_results.csv",
    "sgd_cv_fold_results.csv",
    "xgb_cv_fold_results.csv"
]

for file in csv_files:
    df = pd.read_csv(file)

    # Standard deviation across folds
    acc_sd = np.std(df["accuracy"], ddof=1) # ddof=1 → sample std (important)
    f1_sd = np.std(df["macro_f1"], ddof=1)

    supervised_mean_accuracy_sd.append(acc_sd)
    supervised_mean_f1_sd.append(f1_sd)

print("Accuracy SD:", supervised_mean_accuracy_sd)
print("F1 SD:", supervised_mean_f1_sd)
```

```
Accuracy SD: [0.000611552908386723, 0.002463908733275869, 0.002938203260243771, 0.002139899549754119, 0.0014810722244768997, 0.0034851966024922656]
F1 SD: [3.941845417560423e-06, 0.005194525536461774, 0.005628427258745979, 0.002033437885311681, 0.0030560785266194536, 0.0032156308276492874]
```

```
In [ ]: # Make a bar plot showing accuracy and f1score in red and blue respectively
# Example positions
x = np.arange(len(model_label))
width = 0.35 # width of each bar

# Accuracy bars (red)
plt.figure(figsize=(10,10))
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.bar(x - width/2, supervised_mean_accuracy,
        width, yerr=supervised_mean_accuracy_sd,
        color='red', alpha=0.7, label='Accuracy',
```

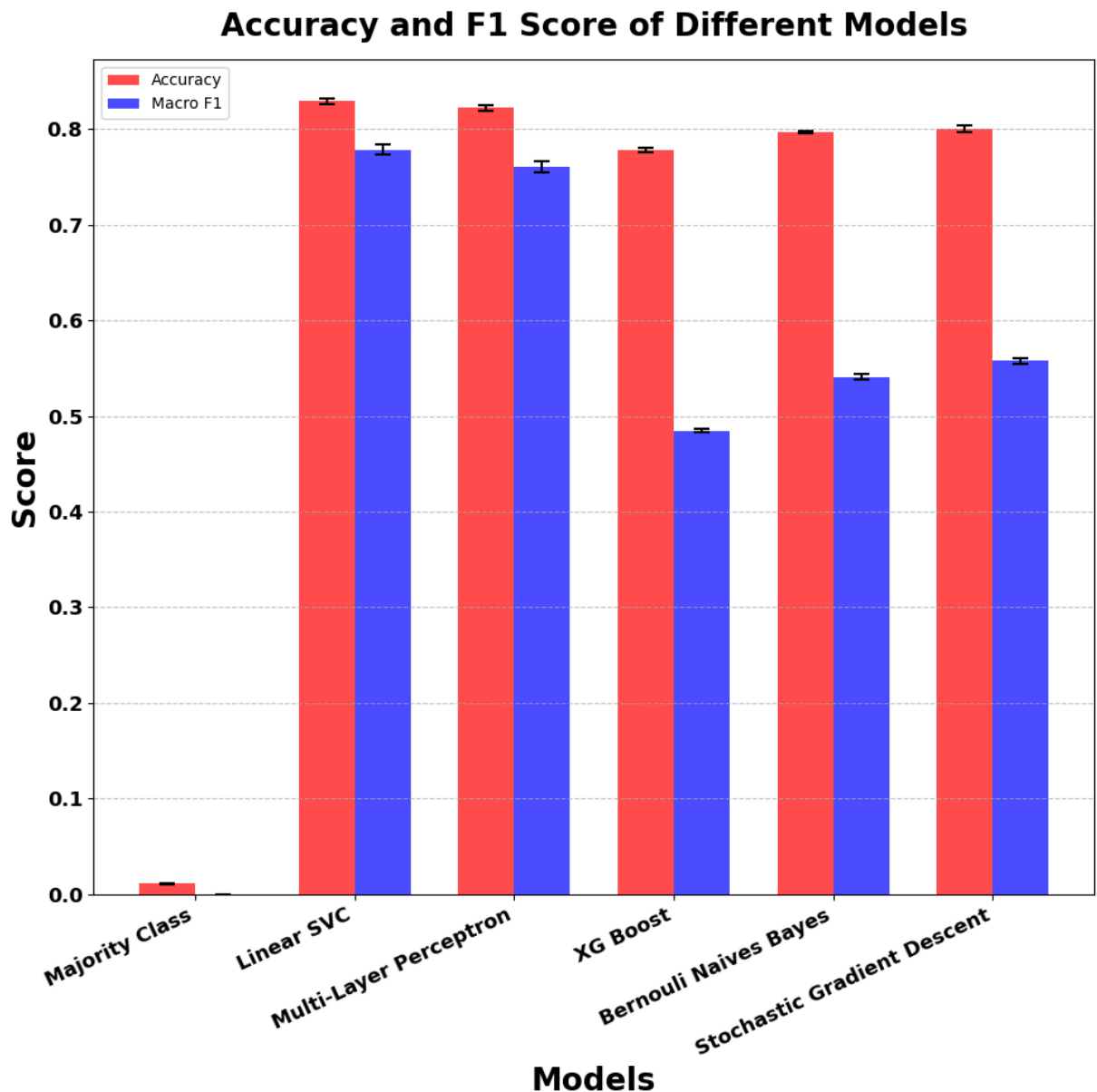
```

        capsize=5, error_kw={'elinewidth':1.5, 'capthick':1.5})

plt.bar(x + width/2, supervised_mean_f1,
        width, yerr=supervised_mean_f1_sd,
        color='blue', alpha=0.7, label='Macro F1',
        capsize=5, error_kw={'elinewidth':1.5, 'capthick':1.5})

# Labels
# make x axis label bold
plt.title('Accuracy and F1 Score of Different Models',
          fontsize=20, fontweight='bold', pad=15)
plt.xlabel('Models', fontsize=20, fontweight='bold')
plt.ylabel('Score', fontsize=20, fontweight='bold')
plt.xticks(x, model_label, rotation=25, ha='right', fontsize=13, fontweight=
plt.yticks(fontsize=13, fontweight='bold')
plt.legend(loc='upper left')
plt.tight_layout() # prevents label cutoff
plt.show()

```



```
In [ ]: !pip install glob
```

```
ERROR: Could not find a version that satisfies the requirement glob (from ve  
rsions: none)  
ERROR: No matching distribution found for glob
```

```
In [ ]: import os  
import glob  
import numpy as np  
import pandas as pd  
from scipy import stats  
from statsmodels.stats.multicomp import pairwise_tukeyhsd  
import statsmodels.api as sm  
from statsmodels.formula.api import ols  
from statsmodels.stats.libqsturng import qsturng  
  
# -----  
# 1. Load and combine all CV result CSVs  
# -----  
  
csv_files = glob.glob("*_cv_fold_results.csv")  
  
dfs = []  
for file in csv_files:  
    df = pd.read_csv(file)  
  
    # If you want cleaner model names from filenames:  
    # bernnb_cv_fold_results.csv -> BernoulliNB  
    # dummy_cv_fold_results.csv  -> Dummy  
    # linear_svc_cv_fold_results.csv -> LinearSVC  
    # etc.  
    filename_model = os.path.basename(file).replace("_cv_fold_results.csv",  
  
    # Use the model column if present, otherwise fallback to filename  
    if "model" not in df.columns:  
        df["model"] = filename_model  
  
    # Create a cleaner label based on filename  
    model_map = {  
        "bernnb": "BernoulliNB",  
        "dummy": "Dummy",  
        "linear_svc": "LinearSVC",  
        "mlp": "MLP",  
        "sgd": "SGD",  
        "xgb": "XGBoost"  
    }  
    df["model_label"] = model_map.get(filename_model, filename_model)  
  
    dfs.append(df)  
  
all_df = pd.concat(dfs, ignore_index=True)  
  
# Keep only needed columns  
all_df = all_df[["model_label", "fold", "accuracy", "macro_f1"]].copy()  
  
print("Combined dataframe:")
```

```

print(all_df.head())
print("\nModels found:", all_df["model_label"].unique())

# -----
# 2. Function to run ANOVA + Tukey HSD/Tukey-Kramer
# -----

def run_anova_tukey(df, metric, alpha=0.05):
    """
    Runs one-way ANOVA and Tukey HSD on the given metric.
    Outputs:
        - ANOVA table
        - Tukey summary table
        - Custom pairwise table with abs mean diff, HSD, q-statistic, reject
    """
    print("\n" + "="*80)
    print(f"ANALYSIS FOR: {metric.upper()}")
    print("="*80)

    # -----
    # Descriptive statistics
    # -----
    summary = df.groupby("model_label")[metric].agg(["mean", "std", "count"])
    summary["se"] = summary["std"] / np.sqrt(summary["count"])
    print("\nDescriptive statistics:")
    print(summary)

    # -----
    # One-way ANOVA
    # -----
    model = ols(f"{metric} ~ C(model_label)", data=df).fit()
    anova_table = sm.stats.anova_lm(model, typ=2)

    print("\nOne-way ANOVA table:")
    print(anova_table)

    pval = anova_table["PR(>F)"].iloc[0]

    if pval >= alpha:
        print(f"\nANOVA not significant for {metric} (p = {pval:.6g}).")
        print("Skipping Tukey HSD.\n")
        return {
            "summary": summary,
            "anova": anova_table,
            "tukey_summary": None,
            "tukey_custom": None
        }

    print(f"\nANOVA significant for {metric} (p = {pval:.6g}). Running Tukey

    # -----
    # Tukey HSD / Tukey-Kramer
    # -----
    tukey = pairwise_tukeyhsd(
        endog=df[metric],

```

```

        groups=df["model_label"],
        alpha=alpha
    )

    print("Tukey HSD summary:")
    print(tukey.summary())

    # Convert Tukey result to DataFrame
    tukey_df = pd.DataFrame(
        data=tukey._results_table.data[1:],
        columns=tukey._results_table.data[0]
    )

    # -----
    # Build custom output table
    # -----
    # Need:
    # - pair
    # - absolute difference of means
    # - HSD threshold
    # - q-statistic used
    # - reject or not
    #
    # For equal n, Tukey HSD threshold = q_crit * sqrt(MSE / n)
    # More generally, statsmodels handles Tukey-Kramer internally.
    # We'll compute pairwise q statistic using:
    #  $q = |\text{mean}_i - \text{mean}_j| / \sqrt{\text{MSE}/2 * (1/n_i + 1/n_j)}$ 
    # HSD threshold (pair-specific Tukey-Kramer threshold):
    #  $\text{HSD}_{\text{pair}} = q_{\text{crit}} * \sqrt{\text{MSE}/2 * (1/n_i + 1/n_j)}$ 
    #
    # reject if abs diff > HSD_pair

    means = df.groupby("model_label")[metric].mean()
    counts = df.groupby("model_label")[metric].count()

    mse = anova_table.loc["Residual", "sum_sq"] / anova_table.loc["Residual", "df"]
    k = df["model_label"].nunique()
    df_error = anova_table.loc["Residual", "df"]

    q_crit = qsturng(1 - alpha, k, df_error)

    custom_rows = []

    for _, row in tukey_df.iterrows():
        g1 = row["group1"]
        g2 = row["group2"]

        mean1 = means[g1]
        mean2 = means[g2]
        n1 = counts[g1]
        n2 = counts[g2]

        abs_diff = abs(mean1 - mean2)

        se_pair = np.sqrt((mse / 2) * (1/n1 + 1/n2))
        q_stat = abs_diff / se_pair

```

```

        hsd_pair = q_crit * se_pair

        custom_rows.append({
            "pair": f"{g1} vs {g2}",
            "mean_1": mean1,
            "mean_2": mean2,
            "abs_mean_diff": abs_diff,
            "q_value_used": q_crit,
            "q_statistic": q_stat,
            "HSD": hsd_pair,
            "reject": abs_diff > hsd_pair
        })

    custom_df = pd.DataFrame(custom_rows)

    print("\nCustom Tukey table:")
    print(custom_df)

    return {
        "summary": summary,
        "anova": anova_table,
        "tukey_summary": tukey_df,
        "tukey_custom": custom_df
    }

# -----
# 3. Run for ACCURACY
# -----

accuracy_results = run_anova_tukey(all_df, metric="accuracy", alpha=0.05)

# -----
# 4. Run for MACRO F1
# -----

f1_results = run_anova_tukey(all_df, metric="macro_f1", alpha=0.05)

# -----
# 5. Optional: save outputs to CSV
# -----

if accuracy_results["summary"] is not None:
    accuracy_results["summary"].to_csv("anova_accuracy_descriptive_stats.csv")
    accuracy_results["anova"].to_csv("anova_accuracy_table.csv")
    if accuracy_results["tukey_summary"] is not None:
        accuracy_results["tukey_summary"].to_csv("tukey_accuracy_summary.csv")
    if accuracy_results["tukey_custom"] is not None:
        accuracy_results["tukey_custom"].to_csv("tukey_accuracy_custom.csv", inc

if f1_results["summary"] is not None:
    f1_results["summary"].to_csv("anova_macro_f1_descriptive_stats.csv")
    f1_results["anova"].to_csv("anova_macro_f1_table.csv")
    if f1_results["tukey_summary"] is not None:
        f1_results["tukey_summary"].to_csv("tukey_macro_f1_summary.csv", inc

```



```
if f1_results["tukey_custom"] is not None:  
    f1_results["tukey_custom"].to_csv("tukey_macro_f1_custom.csv", index=False)  
print("\nDone. CSV outputs saved.")
```

Combined dataframe:

	model_label	fold	accuracy	macro_f1
0	MLP	1	0.824547	0.755898
1	MLP	2	0.825848	0.761688
2	MLP	3	0.822997	0.766032
3	MLP	4	0.821237	0.764876
4	MLP	5	0.818336	0.753115

Models found: ['MLP' 'LinearSVC' 'BernoulliNB' 'SGD' 'Dummy' 'XGBoost']

=====
=====
ANALYSIS FOR: ACCURACY
=====

Descriptive statistics:

	mean	std	count	se
model_label				
BernoulliNB	0.797105	0.002140	5	0.000957
Dummy	0.010753	0.000612	5	0.000273
LinearSVC	0.829135	0.002464	5	0.001102
MLP	0.822593	0.002938	5	0.001314
SGD	0.800756	0.001481	5	0.000662
XGBoost	0.778519	0.003485	5	0.001559

One-way ANOVA table:

	sum_sq	df	F	PR(>F)
C(model_label)	2.640924	5.0	93216.683583	1.259407e-50
Residual	0.000136	24.0	NaN	NaN

ANOVA significant for accuracy (p = 1.25941e-50). Running Tukey HSD...

Tukey HSD summary:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

=====

group1	group2	meandiff	p-adj	lower	upper	reject

BernoulliNB	Dummy	-0.7864	0.0	-0.791	-0.7817	True
BernoulliNB	LinearSVC	0.032	0.0	0.0274	0.0367	True
BernoulliNB	MLP	0.0255	0.0	0.0208	0.0301	True
BernoulliNB	SGD	0.0037	0.1873	-0.001	0.0083	False
BernoulliNB	XGBoost	-0.0186	0.0	-0.0232	-0.0139	True
Dummy	LinearSVC	0.8184	0.0	0.8137	0.823	True
Dummy	MLP	0.8118	0.0	0.8072	0.8165	True
Dummy	SGD	0.79	0.0	0.7853	0.7947	True
Dummy	XGBoost	0.7678	0.0	0.7631	0.7724	True
LinearSVC	MLP	-0.0065	0.0027	-0.0112	-0.0019	True
LinearSVC	SGD	-0.0284	0.0	-0.033	-0.0237	True
LinearSVC	XGBoost	-0.0506	0.0	-0.0553	-0.046	True
MLP	SGD	-0.0218	0.0	-0.0265	-0.0172	True
MLP	XGBoost	-0.0441	0.0	-0.0487	-0.0394	True
SGD	XGBoost	-0.0222	0.0	-0.0269	-0.0176	True

Custom Tukey table:

d \	pair	mean_1	mean_2	abs_mean_diff	q_value_use
0	BernoulliNB vs Dummy	0.797105	0.010753	0.786352	4.3728
5					
1	BernoulliNB vs LinearSVC	0.797105	0.829135	0.032030	4.3728
5					
2	BernoulliNB vs MLP	0.797105	0.822593	0.025488	4.3728
5					
3	BernoulliNB vs SGD	0.797105	0.800756	0.003651	4.3728
5					
4	BernoulliNB vs XGBoost	0.797105	0.778519	0.018586	4.3728
5					
5	Dummy vs LinearSVC	0.010753	0.829135	0.818382	4.3728
5					
6	Dummy vs MLP	0.010753	0.822593	0.811840	4.3728
5					
7	Dummy vs SGD	0.010753	0.800756	0.790003	4.3728
5					
8	Dummy vs XGBoost	0.010753	0.778519	0.767766	4.3728
5					
9	LinearSVC vs MLP	0.829135	0.822593	0.006542	4.3728
5					
10	LinearSVC vs SGD	0.829135	0.800756	0.028379	4.3728
5					
11	LinearSVC vs XGBoost	0.829135	0.778519	0.050616	4.3728
5					
12	MLP vs SGD	0.822593	0.800756	0.021837	4.3728
5					
13	MLP vs XGBoost	0.822593	0.778519	0.044074	4.3728
5					
14	SGD vs XGBoost	0.800756	0.778519	0.022237	4.3728
5					

	q_statistic	HSD	reject
0	738.678885	0.004655	True
1	30.088415	0.004655	True
2	23.942913	0.004655	True
3	3.429830	0.004655	False
4	17.459192	0.004655	True
5	768.767300	0.004655	True
6	762.621797	0.004655	True
7	742.108714	0.004655	True
8	721.219693	0.004655	True
9	6.145502	0.004655	True
10	26.658585	0.004655	True
11	47.547607	0.004655	True
12	20.513083	0.004655	True
13	41.402105	0.004655	True
14	20.889022	0.004655	True

```

=====
=====
ANALYSIS FOR: MACRO_F1
=====
=====

```

Descriptive statistics:

	mean	std	count	se
model_label				
BernoulliNB	0.540756	0.002033	5	0.000909
Dummy	0.000056	0.000004	5	0.000002
LinearSVC	0.778607	0.005195	5	0.002323
MLP	0.760322	0.005628	5	0.002517
SGD	0.557549	0.003056	5	0.001367
XGBoost	0.484801	0.003216	5	0.001438

One-way ANOVA table:

	sum_sq	df	F	PR(>F)
C(model_label)	1.990263	5.0	28957.32789	1.557127e-44
Residual	0.000330	24.0	NaN	NaN

ANOVA significant for macro_f1 (p = 1.55713e-44). Running Tukey HSD...

Tukey HSD summary:

Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
BernoulliNB	Dummy	-0.5407	0.0	-0.5479	-0.5334	True
BernoulliNB	LinearSVC	0.2379	0.0	0.2306	0.2451	True
BernoulliNB	MLP	0.2196	0.0	0.2123	0.2268	True
BernoulliNB	SGD	0.0168	0.0	0.0095	0.024	True
BernoulliNB	XGBoost	-0.056	0.0	-0.0632	-0.0487	True
Dummy	LinearSVC	0.7786	0.0	0.7713	0.7858	True
Dummy	MLP	0.7603	0.0	0.753	0.7675	True
Dummy	SGD	0.5575	0.0	0.5502	0.5647	True
Dummy	XGBoost	0.4847	0.0	0.4775	0.492	True
LinearSVC	MLP	-0.0183	0.0	-0.0255	-0.011	True
LinearSVC	SGD	-0.2211	0.0	-0.2283	-0.2138	True
LinearSVC	XGBoost	-0.2938	0.0	-0.3011	-0.2866	True
MLP	SGD	-0.2028	0.0	-0.21	-0.1955	True
MLP	XGBoost	-0.2755	0.0	-0.2828	-0.2683	True
SGD	XGBoost	-0.0727	0.0	-0.08	-0.0655	True

Custom Tukey table:

d \	pair	mean_1	mean_2	abs_mean_diff	q_value_use
0	BernoulliNB vs Dummy	0.540756	0.000056	0.540699	4.3728
5					
1	BernoulliNB vs LinearSVC	0.540756	0.778607	0.237851	4.3728
5					
2	BernoulliNB vs MLP	0.540756	0.760322	0.219566	4.3728
5					
3	BernoulliNB vs SGD	0.540756	0.557549	0.016793	4.3728
5					
4	BernoulliNB vs XGBoost	0.540756	0.484801	0.055955	4.3728
5					
5	Dummy vs LinearSVC	0.000056	0.778607	0.778551	4.3728
5					
6	Dummy vs MLP	0.000056	0.760322	0.760265	4.3728
5					

7	Dummy vs SGD	0.000056	0.557549	0.557493	4.3728
5					
8	Dummy vs XGBoost	0.000056	0.484801	0.484744	4.3728
5					
9	LinearSVC vs MLP	0.778607	0.760322	0.018285	4.3728
5					
10	LinearSVC vs SGD	0.778607	0.557549	0.221058	4.3728
5					
11	LinearSVC vs XGBoost	0.778607	0.484801	0.293806	4.3728
5					
12	MLP vs SGD	0.760322	0.557549	0.202773	4.3728
5					
13	MLP vs XGBoost	0.760322	0.484801	0.275521	4.3728
5					
14	SGD vs XGBoost	0.557549	0.484801	0.072748	4.3728
5					

	q_statistic	HSD	reject
0	326.099323	0.007251	True
1	143.449745	0.007251	True
2	132.421688	0.007251	True
3	10.128132	0.007251	True
4	33.746767	0.007251	True
5	469.549068	0.007251	True
6	458.521010	0.007251	True
7	336.227454	0.007251	True
8	292.352555	0.007251	True
9	11.028057	0.007251	True
10	133.321614	0.007251	True
11	177.196513	0.007251	True
12	122.293556	0.007251	True
13	166.168455	0.007251	True
14	43.874899	0.007251	True

Done. CSV outputs saved.